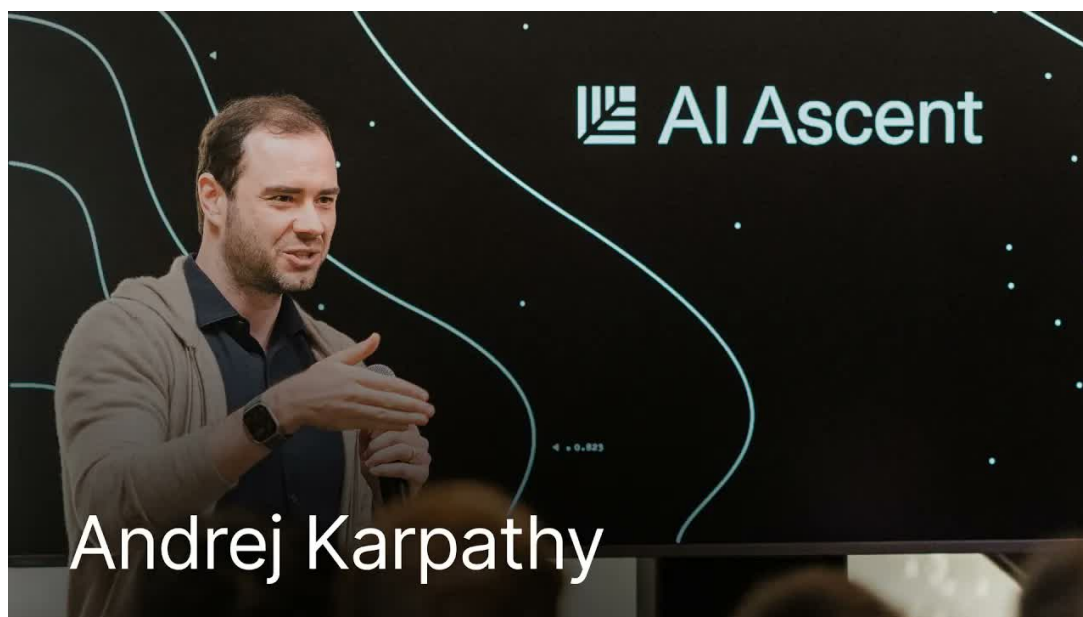


课程笔记

# Andrej Karpathy: 从 Vibe Coding 到 Agentic Engineering

基于 Sequoia Capital AI Ascent 访谈整理

Codex 多智能体流程生成



视频标题: Andrej Karpathy: From Vibe Coding to Agentic Engineering

频道: Sequoia Capital

发布时间: 2026-04-29

视频时长: 29:49

链接: <https://www.youtube.com/watch?v=96jN20C0fLs>

## 目录

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>1</b> | <b>为什么资深程序员也会“落后”：从工具好用到范式迁移</b>                        | <b>3</b>  |
| 1.1      | 从“补全代码”到“连续交付”                                          | 3         |
| 1.2      | 信任阈值：什么时候人开始放手                                          | 4         |
| 1.3      | vibe coding 作为现象，而非完整工程方法                               | 5         |
| 1.4      | 本章小结                                                    | 6         |
| <b>2</b> | <b>软件 1.0/2.0/3.0：上下文窗口成为新的程序接口</b>                     | <b>6</b>  |
| 2.1      | 三代软件范式：代码、权重、上下文                                        | 6         |
| 2.2      | LLM 作为解释器：prompt 与 context window 的杠杆                   | 8         |
| 2.3      | 安装脚本案例：从 bash 到交给 agent 的技能文本                           | 8         |
| 2.4      | 本章小结                                                    | 10        |
| <b>3</b> | <b>新机会来自信息重编译：MenuGen 与原生神经计算</b>                       | <b>10</b> |
| 3.1      | MenuGen 的旧范式：OCR、生成图、重排界面                               | 10        |
| 3.2      | 一条 prompt 的新范式：图片进、图片出                                  | 11        |
| 3.3      | 从结构化数据到非结构化信息重编译                                        | 12        |
| 3.4      | 神经网络作为主进程，CPU 作为协处理器                                    | 13        |
| 3.5      | 本章小结                                                    | 14        |
| <b>4</b> | <b>可验证性如何塑造能力：RL 环境、锯齿智能与能力分布</b>                       | <b>15</b> |
| 4.1      | 传统自动化看“能否写成代码”，LLM 自动化看“能否给出奖励”                         | 15        |
| 4.2      | RL 环境如何把能力推高                                            | 15        |
| 4.3      | 锯齿智能：高峰与凹谷并存                                            | 17        |
| 4.4      | 数据分布与实验室偏好：为什么有些能力突然跃迁                                  | 19        |
| 4.5      | 本章小结                                                    | 21        |
| <b>5</b> | <b>创始人该找什么：在“可验证但未被充分覆盖”的区域建环境</b>                      | <b>21</b> |
| 5.1      | 可验证性让问题变得 tractable                                     | 21        |
| 5.2      | 创业机会：自建数据集、任务集与 RL 环境                                   | 23        |
| 5.3      | 自动化的远近：从确定性验证到 LLM judge                                | 24        |
| 5.4      | 从环境到可靠交付                                                | 25        |
| 5.5      | 本章小结                                                    | 25        |
| <b>6</b> | <b>从 vibe coding 到 agentic engineering：速度、质量与责任同时成立</b> | <b>25</b> |
| 6.1      | vibe coding 提升地板                                        | 25        |
| 6.2      | agentic engineering 维护质量线                               | 27        |
| 6.3      | 速度上限：从 10x 到更大的差距                                       | 28        |
| 6.4      | AI-native 工程师的工作方式                                      | 28        |
| 6.5      | 招聘与评估：从谜题到大项目攻防                                         | 29        |
| 6.6      | 本章小结                                                    | 31        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>7 人类仍负责什么：spec、taste、fundamentals 与 agent-native 世界</b> | <b>31</b> |
| 7.1 agent 像实习生：强记忆、弱判断、会犯奇怪错误 . . . . .                    | 31        |
| 7.2 spec 与 design taste：从用户 ID 错误到系统边界 . . . . .           | 32        |
| 7.3 API 细节可以外包，底层模型不能外包 . . . . .                          | 33        |
| 7.4 为什么美学和简洁仍在 RL 回路之外 . . . . .                           | 34        |
| 7.5 agent-native 基础设施：面向 agent 的文档、权限、传感器与执行器 . . . . .    | 35        |
| 7.6 本章小结 . . . . .                                         | 36        |
| <b>8 综合章：廉价智能时代，不能外包的是理解</b>                               | <b>36</b> |
| 8.1 你可以外包思考过程，但不能外包理解 . . . . .                            | 37        |
| 8.2 知识库与投影：用 LLM 增强理解 . . . . .                            | 37        |
| 8.3 两条主线的汇合：verifier 让模型变强，spec 让团队变强 . . . . .            | 39        |
| 8.4 speaker closing discussion：教育、瓶颈与未来图景 . . . . .        | 39        |
| 8.5 面向后续实践的清单 . . . . .                                    | 40        |
| 8.6 本章小结 . . . . .                                         | 41        |

# 1 为什么资深程序员也会“落后”：从工具好用到范式迁移

这场访谈的开场并没有先讲模型架构、benchmark 或产品发布,而从一个很个人的感受切入:Karpathy 说自己从未像最近这样感觉“作为程序员落后”。这句话刺耳,原因很简单:说这话的人参与过现代 AI 的建造,也长期把复杂技术讲得清楚。他的“落后感”并非来自不会写代码,真正来源是工具边界突然向外扩张,原来需要人反复纠错、拼接、调试的工作块,开始被模型连续完成。

这个开场有教学价值。它把本讲义的核心问题压缩成一句话:当工具从“偶尔帮忙”变成“可以委派”,程序员的工作方式要怎样重校准?

## 1.1 从“补全代码”到“连续交付”

Karpathy 回忆,过去一年他已经在使用各类智能体工具 (agentic tools)。按照本讲义的工作定义,智能体工具指能读取上下文、调用工具、执行多步任务并根据反馈继续推进的软件系统。它们与早期聊天机器人或简单代码补全的关键差别在于:它们有更强的行动半径,可以把“读文件、改代码、跑命令、看错误、再修正”连成一个工作循环。

早期体验更像“局部增强”:模型能写出一段函数、补一个测试、解释一个 API,但输出经常需要人逐行修补。人类仍像主驾驶,模型像副驾驶递来零件。Karpathy 说,到 2024 年 12 月前后,他观察到一个明显转折:最新模型生成的代码块“出来就是好的”,继续要求更多,也仍然可用;他甚至记不起上一次认真纠正模型输出是什么时候。这里的重点不只是模型单次回答更强,更是连续任务中的连贯性明显提升。

**2024 年底的分水岭**

这个时间点在访谈中承担的是经验转折,而非严格统计结论。Karpathy 的意思是:许多人如果只保留了早期 ChatGPT 式体验,会低估后续 agentic workflow 的变化。模型从“能写片段”走到“能连续推进工作块”,人的使用姿态会随之改变。

智能体工作流 (agentic workflow) 就是这种使用姿态的名字:由人提出目标,agent 分解、执行、观察结果、修正方案,形成多轮循环。它与一次性 prompt 的区别像“给一个人发一条消息”和“让一个人带着反馈做完一件事”。前者的核心是回答质量,后者的核心是任务闭环。



图 1: Karpathy 描述 2024 年底前后代码生成体验的质变：模型开始连续完成更大的可用工作块，vibe coding 因此从边缘体验进入主流讨论。<sup>2</sup>

### 1.2 信任阈值：什么时候人开始放手

真正改变行为的并非平均质量的一点点提升，关键在于人对系统的校准发生了相变。信任阈值（trust threshold）指模型连续产出达到某个可用率后，人类从逐行纠错转向委派更大任务的心理与工程阈值。它有两层含义。

第一层是心理阈值。只要人预期“十行里总有三行要改”，使用者就会保持紧张，随时准备接管。模型即使有用，也很难承担完整任务。第二层是工程阈值。只要输出能连续通过本地检查、测试、运行和人工快速审阅，任务粒度就会自然变大：从“帮我写一个函数”变成“把这个功能接上去”，再变成“做一个小项目原型”。

#### 核心定义：信任阈值

信任阈值（trust threshold）是模型连续产出达到某个可用率后，人类从逐行纠错转向委派更大任务的心理与工程阈值。它区别于盲目信任；它是一种基于反复体验形成的校准：人发现纠错频率下降，于是愿意把更大的任务块交给系统。

这也解释了“感觉落后”的来源。人类能力没有突然退化，参照系却变了。过去熟练程序员的优势在于知道语法、库、脚手架、调试路径和工程套路；现在其中一部分被 agent 吸收成可快速调用的行动能力。熟练程序员仍然重要，但“熟练”本身的含义在改变：更少体现在逐行敲代码，更多体现在提出目标、约束范围、识别风险和验收结果。

<sup>2</sup> 视频画面时间区间：00:01:38–00:01:45。

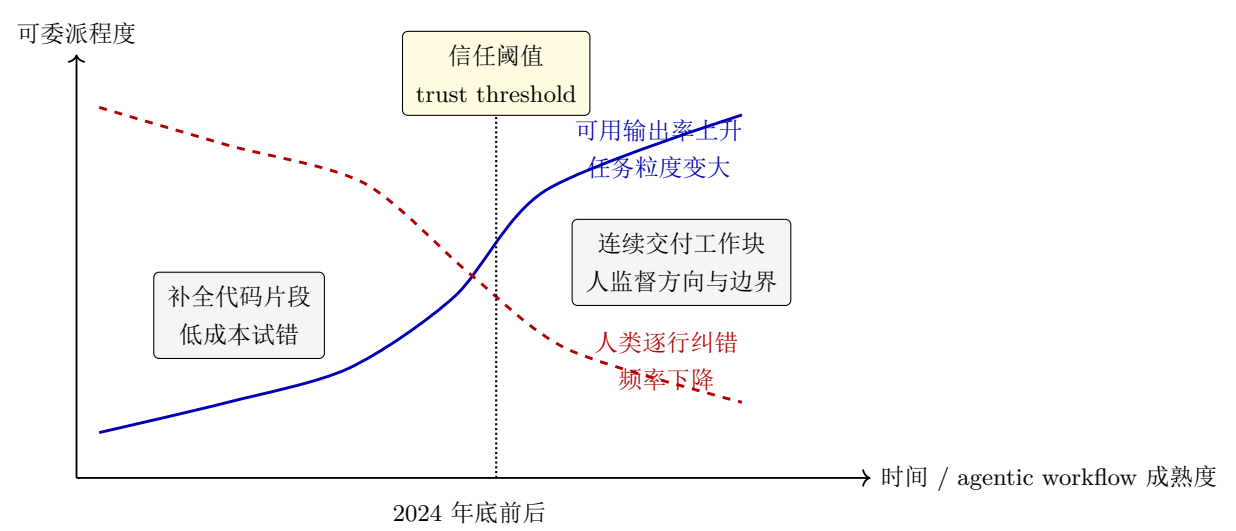


图 2: 信任阈值示意图：当模型连续产出的可用率越过某个阈值，人会把交互方式从逐步纠错转向委派更大的工作块。作者根据访谈内容绘制。

信任阈值区别于自动化偏见

越过信任阈值不等于把判断力交出去。自动化偏见会让人因为“机器看起来顺畅”就放弃检查；健康的信任阈值需要反复校准、局部验证和清晰的失败边界。后文的 agentic engineering 正是为了解决这个问题。

1.3 vibe coding 作为现象，而非完整工程方法

Karpathy 在这里自然引出了 vibe coding。按照本讲义的工作定义，vibe coding 是人用自然语言和快速反馈驱动软件创作，常以信任模型输出和低摩擦试错为特征。它像把软件创作从“精确雕刻”暂时切到“快速塑形”：人描述想要的感觉、功能或变化，agent 生成，人在结果上继续反馈。

这个词之所以流行，是因为它抓住了一种新体验：软件创作的地板被抬高了。side projects 变成低成本试验场，很多过去会因为脚手架、环境配置、库选择而搁置的小想法，如今可以迅速变成可运行的雏形。Karpathy 提到自己的 side projects 文件夹堆满了随机项目，这个细节指向范式变化的早期症状：当试错成本下降，想法数量会迅速膨胀。

核心定义：vibe coding

vibe coding 是一种由自然语言、模型生成和快速反馈驱动的软件创作方式。它适合探索、原型、个人项目和低成本试验；进入生产环境时，还需要测试、审计、规格说明、安全边界和人工判断来承接。

因此，本章不能把 vibe coding 写成终点。它更像入口词：它让更多人感受到“软件 3.0”的威力，也暴露出下一阶段的问题。原型能跑起来只是第一步；一个系统能长期维护、能处理异常、能抵抗误用、能被团队理解，才进入工程世界。后文会把这条线升级到 agentic engineering，也就是用工程纪律协调 agent，在速度、质量、安全和责任之间建立可操作的平衡。



## 1.4 本章小结

本章用 Karpathy 的“落后感”打开全篇：落后感的真正含义是工具边界快速外扩，人的工作方式需要重新校准。2024 年底前后的关键变化在于 agentic workflow 的连贯性增强，模型从局部代码辅助走向连续任务推进。

本章完整定义了四个基础术语：智能体工具（agentic tools）、智能体工作流（agentic workflow）、信任阈值（trust threshold）和 vibe coding。它们共同说明一件事：当模型连续产出的可用率跨过信任阈值，人类会自然扩大委派粒度，side projects 会爆发，软件创作地板会被抬高。

但这只是第一幕。vibe coding 让人更快开始，不能保证系统更可靠。讲义后续要回答的尖锐问题是：怎样把这种低摩擦创造力，升级成可审计、可维护、可验证、能进入真实生产环境的工程能力。

## 2 软件 1.0/2.0/3.0：上下文窗口成为新的程序接口

上一章解释了为什么 Karpathy 会突然更信任 agent：输出质量和连续工作流越过了信任阈值。本章往下挖一层，回答“为什么这已经超出更好的代码助手”。Karpathy 的说法是，LLM 像一种新计算机，软件正在经历 1.0、2.0、3.0 的范式迁移。这里的“软件版本号”指向人类控制计算方式的三种杠杆。

### 2.1 三代软件范式：代码、权重、上下文

先给直觉：如果把计算看成从输入到输出的转换，三代软件范式的差别在于人把“控制信息”放在哪里。软件 1.0 把控制信息写进代码；软件 2.0 把控制信息压进训练后的权重；软件 3.0 把控制信息组织进上下文窗口，让 LLM 解释后执行。

软件 1.0（Software 1.0）指人直接写显式规则和程序逻辑，执行主体是传统计算机。它的优势是确定、可审计、边界清楚；代价是所有分支都要被程序员提前想到。shell 脚本、Web 后端、数据库迁移、编译器中的规则，大多属于这个世界。

软件 2.0（Software 2.0）指人通过数据集、目标函数、网络架构训练权重，让模型学习行为。这里的权重（learned weights）是训练后固化在模型中的参数，承载从数据中学到的统计结构和技能。自动驾驶感知、图像分类、语音识别、早期深度学习系统，典型地属于这个范式：人减少逐条规则编写，把主要精力转向训练过程设计。

软件 3.0（Software 3.0）指人通过 prompt、上下文和工具环境来“编程”LLM，让模型解释上下文并执行信息空间中的计算。它把“当前要做什么、有哪些资料、运行环境是什么、刚刚出错了什么”放进上下文窗口，再让模型基于这些信息行动。

#### 三代范式的工作定义

软件 1.0 用显式代码控制计算；软件 2.0 用数据、目标函数和网络架构训练权重；软件 3.0 用 prompt、上下文窗口和工具环境来驱动 LLM 解释并行动。三者会长期共存，工程系统通常会把它们组合起来。

用一个教学公式可以压缩三代范式。直觉上，同样是从输入  $x$  得到输出  $y$ ，三代范式改变的是“人通过什么介质影响结果”。

$$S_1 : y = f_{\text{code}}(x), \quad S_2 : y = M_W(x), \quad W = \text{Train}(D_{\text{train}}, L), \quad S_3 : y = M_W(C)$$

-  $S_1$ : 软件 1.0, 显式代码控制计算。-  $S_2$ : 软件 2.0, 训练数据和损失函数塑造权重。-  $S_3$ : 软件 3.0, 上下文窗口成为主要控制面。-  $x$ : 输入, 例如用户请求、图片、日志或业务数据。-  $y$ : 输出, 例如程序结果、模型回答、代码补丁或行动建议。-  $f_{code}$ : 人写出的传统程序。-  $M_W$ : 由权重  $W$  参数化的模型。-  $W$ : 模型权重, 即训练后固化在模型中的参数。-  $D_{train}$ : 训练数据分布。-  $L$ : 训练目标或损失函数。-  $C$ : 上下文窗口中的任务、资料、环境观察和工具结果。



图 3: 访谈进入 Software 3.0 框架: 程序的控制面从显式代码、训练权重, 扩展到 prompt、上下文窗口与工具环境。<sup>4</sup>

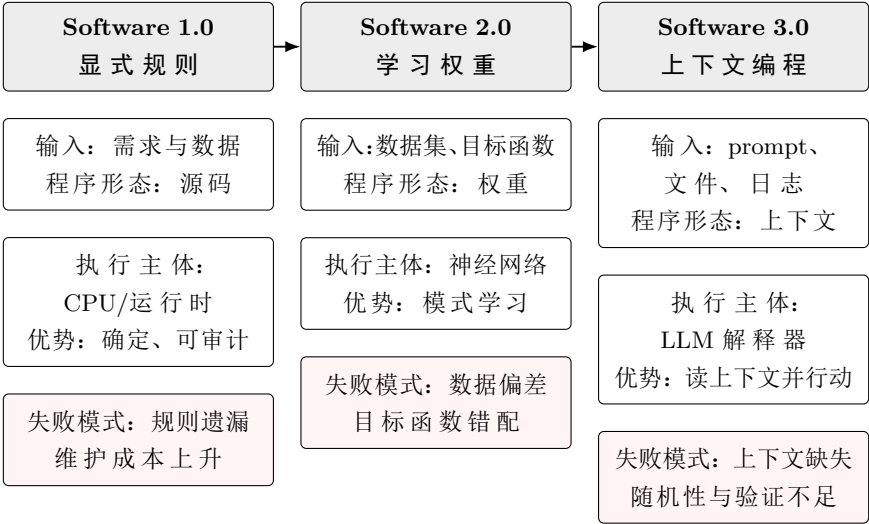


图 4: Software 1.0/2.0/3.0 对照图: 三者分别把控制权放在显式代码、训练出来的权重、以及可被 LLM 解释的上下文中。作者根据访谈内容绘制。

<sup>4</sup>视频画面时间区间: 00:02:28-00:02:40。



## 2.2 LLM 作为解释器：prompt 与 context window 的杠杆

Karpathy 说，软件 3.0 中的编程转向 prompting，而上下文窗口是人控制解释器的杠杆。上下文窗口（context window）指模型一次推理可读取的文本、代码、图像、工具结果等信息范围，是软件 3.0 的主要控制面。它像一张临时工作台：你把任务说明、相关文件、错误日志、运行结果、约束条件放上去，模型就在这张工作台上组织推理和行动。

LLM 解释器（LLM as interpreter）是一个有用类比：把 LLM 看成解释上下文并产生行动或输出的执行体，类似解释器读取程序文本。这个类比不能拉得太满。传统解释器通常按形式语义逐步执行，LLM 的输出带概率性，会受训练分布、采样、上下文排列和工具反馈影响。类比的价值在于提醒读者：在软件 3.0 中，prompt 和上下文已经超出“聊天内容”，开始承担程序接口的角色。

### 上下文窗口像临时工作台

工作台上放什么，助手就能看见什么；工作台太乱，助手会漏掉关键约束；工作台缺少错误日志，助手只能猜。软件 3.0 的工程能力很大程度来自上下文管理：选什么资料、删什么噪声、保留哪些观察、怎样让模型知道当前目标。

工具使用（tool use）指模型调用外部命令、API、浏览器、数据库、编辑器等能力来扩展自身行动范围。没有工具时，LLM 主要在文本中推演；有工具时，它可以运行命令、读取文件、检查错误、修改代码、再次验证。工具集合  $T$  像一组执行器，行动序列  $a_{1:k}$  则是 agent 在 multi-step 工作流中的实际操作轨迹。

直觉上，软件 3.0 的一次任务不只产生一个答案，还可能产生一串动作。上下文会随着动作观察不断更新：

$$S_3 : (C, M_W, T) \rightarrow y \text{ or } a_{1:k}, \quad C_{t+1} = C_t \cup \text{obs}(a_t)$$

-  $S_3$ ：软件 3.0 范式。-  $C$ ：当前上下文窗口。-  $M_W$ ：由权重  $W$  参数化的 LLM。-  $T$ ：工具集合，例如 shell、浏览器、数据库、编辑器或部署平台。-  $y$ ：最终输出，例如解释、代码补丁或配置结果。-  $a_{1:k}$ ：从第 1 步到第  $k$  步的行动序列。-  $a_t$ ：第  $t$  步行动。-  $C_t$ ：第  $t$  步前模型可见的上下文。-  $\text{obs}(a_t)$ ：执行第  $t$  步行动后得到的观察结果，例如命令输出、错误日志或测试结果。

### 不要把 prompt 缩小成一句话

在软件 3.0 中，prompt 只是上下文的一部分。真正起作用的是完整的上下文包：目标、文件、约束、历史、工具结果、错误日志、权限边界。只优化一句提示词，常常解决不了 multi-step 工程任务的失败。

## 2.3 安装脚本案例：从 bash 到交给 agent 的技能文本

Karpathy 给出的安装器例子很直观。传统软件 1.0 世界里，安装工具通常是一段 bash 或 shell 脚本。脚本作者必须预先处理平台差异：不同操作系统、不同 shell、不同包管理器、不同权限、不同路径、不同依赖版本。结果常常是脚本越写越膨胀，充满分支判断和补丁逻辑。

软件 3.0 的思路更像给 agent 一段技能文本：把安装目标、必要步骤、注意事项和检查方式写清楚，让 agent 自己读取环境、执行命令、观察错误、调整方案。脚本作者减少对所有环境分支的枚举，把“如何完成安装”的知识转成 agent 可解释的上下文。这个转变微妙但很大：程序接口从“运行这段确定性脚本”扩展成“把这段操作契约交给一个会调试的执行体”。



图 5: OpenClaw/installer 案例的现场画面: 安装逻辑从人工枚举平台分支, 转向把技能文本交给 agent, 让它读取环境并处理差异。<sup>6</sup>

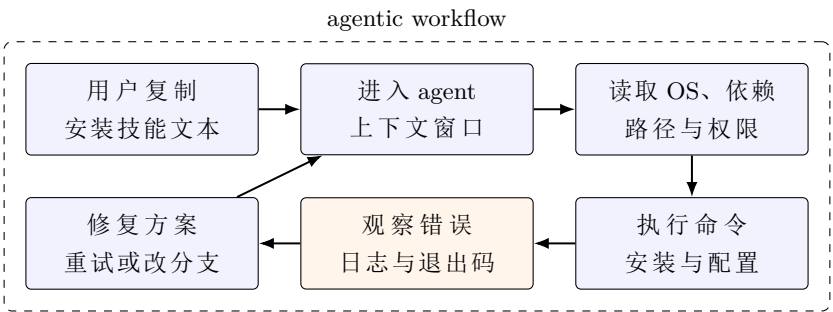


图 6: agent 安装器流程: 技能文本相当于临时程序, LLM 解释上下文并通过工具调用完成读环境、执行、观察、修复的循环。作者根据访谈内容绘制。

这个例子还解释了为什么上一章的 agentic workflow 会突然有用。安装过程天然包含循环: 读取环境, 尝试安装, 失败, 读错误, 改方案, 再尝试。软件 1.0 可以写出这个循环, 但所有分支都要提前编码; 软件 3.0 把许多分支处理交给 LLM 的上下文解释能力和工具使用能力。说得直接一点: 它用模型的通用知识和现场观察, 替代了一部分脚本作者的穷举劳动。

### 安装器案例的核心

软件 3.0 的价值不在于把 bash 脚本换成一段更时髦的文本。它把“安装知识”从显式分支脚本迁移到 agent 可解释的上下文中, 让 agent 利用环境观察和工具使用在循环里完成调试。

当然, 这种做法也有风险。脚本的确定性较强, 行为边界清晰; agent 会根据上下文做判断, 可能执行不合适的命令, 可能误读错误, 也可能遗漏安全约束。因此, 软件 3.0 并不会消灭软件 1.0。更现实的工程结构是: 用软件 3.0 处理开放、杂乱、分支多的任务; 用软件 1.0 的测试、权限、日志和回滚机制守住边界; 用软件 2.0 的权重提供通用能力。

<sup>6</sup> 视频画面时间区间: 00:03:44-00:03:52。

## 2.4 本章小结

本章建立了全篇第一个理论骨架：软件 1.0、2.0、3.0 分别对应代码、权重和上下文三种控制计算的方式。软件 1.0 依赖显式规则；软件 2.0 通过数据和目标函数训练权重；软件 3.0 通过 prompt、上下文窗口和工具环境驱动 LLM 解释并行动。

本章完整定义了软件 1.0 (Software 1.0)、软件 2.0 (Software 2.0)、软件 3.0 (Software 3.0)、上下文窗口 (context window)、LLM 解释器 (LLM as interpreter) 和工具使用 (tool use)。其中最关键的迁移是：上下文窗口开始成为新的程序接口，模型读取上下文、调用工具、观察结果，再把新观察并回上下文。

安装器案例说明了这个范式的工程含义。复杂平台差异过去需要脚本作者提前枚举；现在可以把一段技能文本交给 agent，让它读环境、试命令、看错误、修正路径。这个能力很强，也很锋利。后文讨论 agentic engineering 时，必须补上测试、权限、审计、回滚和人类判断，否则速度会变成风险放大器。

## 3 新机会来自信息重编译：MenuGen 与原生神经计算

### 3.1 MenuGen 的旧范式：OCR、生成图、重排界面

Karpathy 讲 MenuGen 时，表面上是在讲一个餐厅菜单小应用：用户拍一张菜单照片，系统识别菜名，再为每道菜生成一张大致长相的图片，最后把结果重新排版给用户看。它的动机非常朴素：菜单经常没有图片，用户面对陌生菜名时缺少感性参照，点餐像在读一份没有配图的 API 文档。

按照传统应用开发思路，这个产品会自然分成一串模块：上传图片、OCR、结构化菜品名称、调用图像生成器、保存中间结果、重新渲染菜单界面、部署到 Vercel 或其他平台。每个模块都合理，每个接口都能解释清楚，工程师也能为它画出一张漂亮的流水线图。



图 7: MenuGen 案例被用来展示 Software 3.0 的产品机会：过去需要 OCR、菜品识别、图像生成和界面重排等中间步骤。<sup>8</sup>

<sup>8</sup>视频画面时间区间：00:04:56-00:05:05。

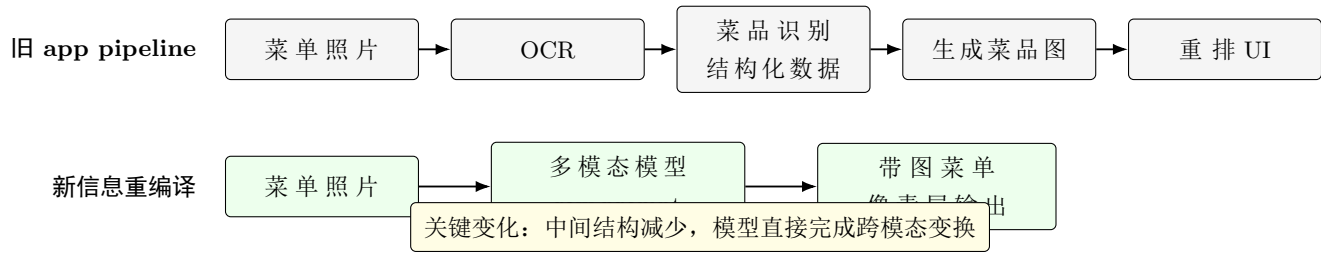


图 8: MenuGen 两种 pipeline 对照：旧方案依赖多个结构化中间层，新方案把菜单照片直接交给多模态模型完成信息重编译。作者根据访谈内容绘制。

OCR 在这里承担什么角色

OCR（Optical Character Recognition，光学字符识别）把图片里的文字转成机器可处理的文本。它像把纸质菜单先抄进电子表格，再让后续程序按表格行列工作。这个步骤很适合传统代码，因为输入输出边界清楚：图片进，文本出。

这条流水线的问题在于，它把“看懂一张菜单并补上视觉线索”拆成了许多中间结构。拆分本身没有错，但它暴露了旧范式的默认假设：计算机擅长处理结构化数据，所以工程师要先把世界切成字段、列表、ID、文件和 UI 组件。MenuGen 的旧实现很像把一道菜先拆成营养成分表，再努力拼回一张能让人产生食欲的照片。

3.2 一条 prompt 的新范式：图片进、图片出

真正让 Karpathy 震动的是另一个版本：直接把菜单照片交给 Gemini，并要求用 Nanobanana 把菜品图像覆盖回原菜单。输入是一张图片，输出仍是一张图片，中间没有显式 OCR 表格、菜品数据库、独立图片卡片和重新渲染界面。模型在像素层完成理解、定位、生成和合成。

多模态模型（multimodal model）指可同时处理文本、图片、音频、视频等多种模态输入或输出的模型。这里的“模态”可以理解为信息的感官通道：文字像说明书，图片像现场照片，音频像录音，视频像带时间轴的观察。传统程序通常要求先把这些通道翻译成统一的数据结构；多模态模型则能直接在多种通道之间建立映射。

多模态模型的工作定义

本章采用的工作定义是：多模态模型可以把图片、文本、音频、视频等多种信息形态放进同一个推理过程，并生成文本、图片或其他形态的结果。MenuGen 的新范式正是“菜单照片进，带菜品图的菜单照片出”。

这一步改变了产品思维。旧范式问：“该怎样搭建一个应用，把菜单识别、图片生成和界面渲染连接起来？”新范式问：“能否把原始信息直接交给模型，让模型输出用户真正想要的形态？”前者的中心是应用架构，后者的中心是信息变换。





图 9: Karpthy 讲到 raw prompt 方案时的转折：把菜单照片和一句 prompt 交给多模态模型，信息变换可以直接发生在像素层。<sup>10</sup>

### 不要只把软件 3.0 想成“旧应用加速器”

如果一个团队只盯着“把 OCR 做快一点、把图片生成做便宜一点、把 UI 重排做自动一点”，它会错过更大的变化：很多中间应用层可能直接被压缩进一次模型推理。旧模块仍有价值，但它们不再天然拥有存在权。

## 3.3 从结构化数据到非结构化信息重编译

Karpthy 随后把 MenuGen 推广到更大的范围：这件事不只关于写代码。过去的代码主要围绕结构化数据运转，例如数据库表、JSON、数组、对象、消息队列。结构化数据像已经切好的积木，程序擅长按规则拼装它们。文档、会议记录、网页、图片、菜单、视频则更像一堆尚未切开的材料，里面有意义，但意义没有整齐地摆进字段。

信息重编译（information recompilation）指把同一批信息重新组织成另一种可用形态。例如，菜单照片变成带菜品图的菜单，组织内部文档变成 wiki，零散事实变成可检索、可问答、可导航的知识库。这里的“重编译”是类比：传统编译器把源代码转换成机器能执行的形态；LLM 和多模态模型把非结构化资料转换成人能行动、组织能协作、agent 能继续处理的形态。

### 信息重编译

信息重编译的核心在于把资料变成另一种可使用的界面，远远超过普通摘要。它把同一份事实重新排列、投影、命名、连接，让原本只能被阅读的材料变成可导航、可查询、可继续加工的对象。

LLM knowledge base 项目就是这个方向的例子。给定一批组织文档，过去很难写出一个传统程序来“理解这些事实并生成一套好用 wiki”。程序可以建索引、抽标题、做关键词搜索，但它很难判断

<sup>10</sup> 视频画面时间区间：00:05:43-00:05:55。

哪些概念应当聚合、哪些页面应当拆分、哪些事实应当作为背景、哪些事实应当放进操作手册。LLM 让这类重排工作进入可自动化范围。



图 10: 通用信息重编译图：Software 3.0 的机会在于把文档、图片、音视频等非结构化材料重排成新的可用形态。作者根据访谈内容绘制。

这也解释了为什么新机会会显得“不像软件”。很多未来产品的核心价值可能不在按钮、表单和 CRUD，而在把原始材料重新生成成某种可用表面：给销售团队的账户地图、给医生的病例时间线、给工程团队的故障知识库、给个人的学习路径。它们都有软件外壳，但真正的计算发生在信息形态之间。

3.4 神经网络作为主进程，CPU 作为协处理器

在 2026 年什么会显得显而易见这个问题上，Karpathy 给出的回答更像未来图景：也许会出现更“神经原生”的计算形态。设备接收原始视频或音频，神经网络根据当下情境用 diffusion 一类生成机制渲染 UI；传统 CPU 与工具被调用来完成确定性任务。这个图景需要谨慎理解，它是方向性推测，尚未成为当前工程现实。

神经计算机（neural computer）在本讲义中指 Karpathy 对未来计算形态的推测：神经网络承担主要信息处理，传统 CPU 或工具承担确定性辅助。主进程（host process）是控制主要流程和资源调度的执行主体；协处理器（co-processor）是被主进程调用来完成专门任务的辅助计算单元。在今天，神经网络通常运行在经典计算机之上；在这个推测中，关系被翻转，神经网络像“运行节奏的指挥台”，CPU 像“负责精确算术、文件操作、协议调用的工作单元”。

这是未来图景，尚非既成事实

“神经网络成为 host process、CPU 成为 co-processor”应写成推测。今天的软件栈仍由操作系统、CPU/GPU、运行时、数据库、网络协议和安全边界支撑。Karpathy 的重点是提示方向：信息处理的重心可能继续向神经网络迁移。

这个想象之所以重要，是因为它把“工具使用”放到新的位置。当前常见说法是模型调用工具来弥补自身不足，例如查数据库、运行测试、执行 shell。神经主进程的图景更进一步：模型负责理解场景、决定界面、组织工作流，把传统计算当作确定性附肢来调用。比喻一下，今天像人坐在电脑前使用 AI 插件；这个未来图景像 AI 坐在控制台前，调用电脑完成精确动作。





图 11: Karpathy 提到 neural computer、host process 与 CPU co-processor 的未来图景；这一段应按方向性推测处理。<sup>12</sup>

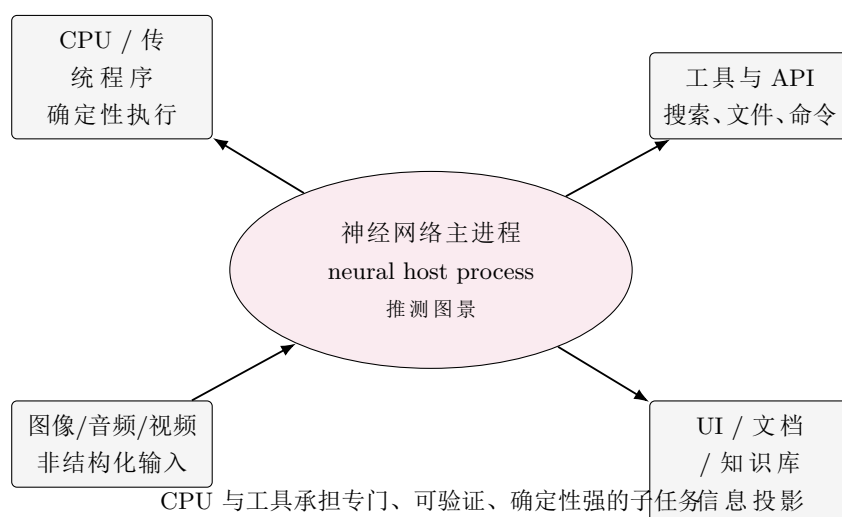


图 12: neural host process 推测图：神经网络可能成为主要的信息调度层，CPU 与工具承担确定性子任务。作者根据访谈内容绘制。

### 3.5 本章小结

MenuGen 的教训很直接：软件 3.0 的机会不止来自把旧流程加速，还来自让旧流程中的中间结构消失。多模态模型把图片、文本、音频、视频纳入同一信息空间，信息重编译把原始材料转换成新的可用表面。

本章需要保留三点：第一，旧 MenuGen 是结构化流水线，新 MenuGen 是原始图片到图片的信息变换；第二，LLM knowledge base 展示了文档到 wiki 的重编译，说明“可计算对象”的边界扩张到非结构化资料；第三，神经计算机、host process 和 co-processor 是 Karpathy 的未来推测，方向值得重视，落地仍需逐步发生。

<sup>12</sup>视频画面时间区间：00:07:37-00:07:50。

## 4 可验证性如何塑造能力：RL 环境、锯齿智能与能力分布

### 4.1 传统自动化看“能否写成代码”，LLM 自动化看“能否给出奖励”

这一章回答一个看似矛盾的问题：为什么同一个前沿模型可以重构十万行代码、发现零日漏洞，却会在“洗车店离我 50 米，我该走路还是开车去洗车”这类常识题上给出荒唐建议？如果只用“模型更聪明了”或“模型还不够聪明”解释，会丢掉真正的机制。

Karpathy 的切入点是可验证性（verifiability）。可验证性指一个任务输出能被自动、稳定、低成本地判断好坏或对错的程度。传统计算机更容易自动化可规格化的任务：规则能写成代码，机器就能执行。LLM 这一轮能力增长更偏向可验证任务：只要系统能给输出打分、判对错、构造反馈，训练就能不断把模型推向更高能力。

#### 可规格化性与可验证性的差别

可规格化性关注“能否把过程写成规则”，可验证性关注“能否判断结果好坏”。数独求解过程可以很复杂，但答案容易检查；一篇散文的生成过程可以很自然，但质量判断成本更高。这个差别解释了很多自动化边界。

直觉上，可以把可验证性看成任务的“自动裁判可得性”。裁判越稳定，训练越容易规模化。为了表达这个直觉，本章使用一个教学符号：

$$V(\tau) \in [0, 1]$$

- $V(\tau)$ ：任务  $\tau$  的可验证性。
- $\tau$ ：一个待完成的问题或工作单元。
- $[0, 1]$ ：教学上的归一化范围，0 表示几乎无法自动判断，1 表示几乎可以稳定自动判定。

代码、数学、形式化推理、漏洞挖掘的许多子任务都有较高可验证性：单元测试能跑，证明能查，漏洞利用能复现，棋局合法性和胜负能判定。开放式战略、审美判断、组织政治、产品取舍则更难构造稳定裁判。

### 4.2 RL 环境如何把能力推高

强化学习环境（reinforcement learning environment）是模型可反复尝试、收到奖励反馈、据此改进策略的任务环境。奖励（reward）是对一次输出或行动结果的数值反馈，表示该行为是否接近目标。验证器（verifier）是自动判定输出质量的程序、测试、规则、模型评审或人工流程。

把三者连起来，能力增长的闭环就出现了：系统给模型一个任务，模型产生输出，验证器打分或判对错，奖励信号回到训练过程，模型在大量试验中强化更有效的行为。这个过程很像训练运动员：教练不需要解释每块肌肉该怎么发力，只要能稳定告诉他每次动作是否更接近目标，他就能通过反复练习进步。



图 13: 访谈转入可验证性框架：LLM 更快自动化那些能够判断输出好坏、能够构造奖励的任务。<sup>13</sup>

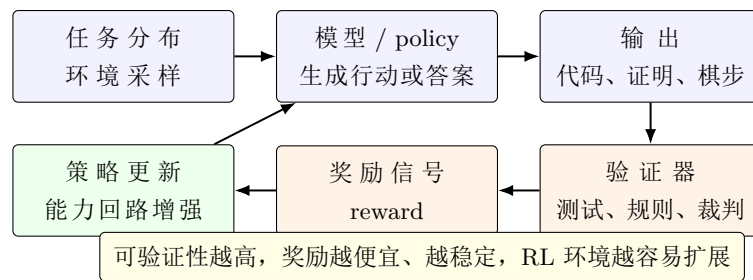


图 14: RL verifier 闭环：任务输出经过验证器转化为奖励，奖励再推动策略更新。作者根据访谈内容绘制。

用符号写出来，RL 环境可以被教学性地表示为：

$$E = (\mathcal{S}, \mathcal{A}, R, P), \quad r = R(\tau, y)$$

- $E$ : 强化学习环境。
- $\mathcal{S}$ : 环境状态集合，例如当前代码库、棋盘局面或题目上下文。
- $\mathcal{A}$ : 模型可采取的动作集合，例如生成答案、修改代码、走一步棋。
- $R$ : 验证器或奖励函数。
- $P$ : 状态转移机制，表示动作如何改变环境状态。
- $r$ : 某次输出得到的奖励。
- $\tau$ : 当前任务。
- $y$ : 模型对任务给出的输出。

<sup>13</sup> 视频画面时间区间：00:09:41–00:09:50。

**可验证性为何会塑造能力峰值**

可验证性提供奖励，奖励让 RL 环境可以扩展，环境扩展带来大量反馈，大量反馈会把模型能力推向峰值。数学和代码之所以增长快，一个关键原因是它们更容易形成这种闭环。

这里的残酷现实是：模型能力并不均匀增长。实验室会优先投入经济价值高、反馈清楚、benchmark 可见的领域。代码正好满足这些条件：价值高、任务多、测试可运行、失败可复现。于是大量训练资源会流向代码相关环境，模型在这些环境中“飞起来”。

**4.3 锯齿智能：高峰与凹谷并存**

锯齿智能（jagged intelligence）指模型能力在不同任务上高低起伏明显：某些专业任务极强，某些常识任务意外薄弱。它像一条不平滑的山脊线，有的地方突然抬升成高峰，有的地方近在旁边却塌成凹谷。

strawberry 例子曾经是典型凹谷：用户问 strawberry 里有几个字母 r，模型会答错。这个任务对人类很简单，因为人可以把单词当字符序列数一遍；模型如果主要按语义和常见文本模式处理，就可能忽略字符级细节。后来模型修补了这个点，但这恰好说明锯齿会随训练与补丁移动。

car wash 例子更有意思：用户说要去 50 米外的洗车店洗车，问该走路还是开车。很多模型会抓住“50 米很近”这个模式，建议走路，却忽略了目标是“洗车”，车必须到场。它在距离常识上表现顺滑，在任务意图上突然断裂。

**锯齿智能要求人留在回路中**

当模型在相邻任务之间表现跳变时，使用者不能把一次强表现推广成全域可靠。它能重构大型代码库，不保证它能处理每个朴素常识约束；它能找漏洞，也不保证它理解“洗车”这个动作需要把车带过去。

能力分布（capability distribution）指模型在不同领域、任务类型、输入形态上的能力水平分布。锯齿智能就是能力分布不平滑。直觉上，若把每个任务放在横轴，把模型成功率或奖励放在纵轴，理想中的通用智能曲线会比较平滑；现实模型更像一排被训练资源、数据覆盖和验证器拉高的尖峰。

$$D_{\text{cap}}(\tau) = \mathbb{E}[R(\tau, M(C_\tau))]$$

- $D_{\text{cap}}(\tau)$ ：模型在任务  $\tau$  上的能力水平。
- $\mathbb{E}[\cdot]$ ：对多次样本、输入变化或评估情形取期望。
- $R$ ：验证器或奖励函数。
- $\tau$ ：任务。
- $M$ ：模型。
- $C_\tau$ ：围绕任务  $\tau$  组织的上下文。



图 15: Karpathy 解释锯齿能力：模型在代码、数学、棋类等可验证领域出现高峰，同时在某些常识任务上出现意外凹谷。<sup>15</sup>

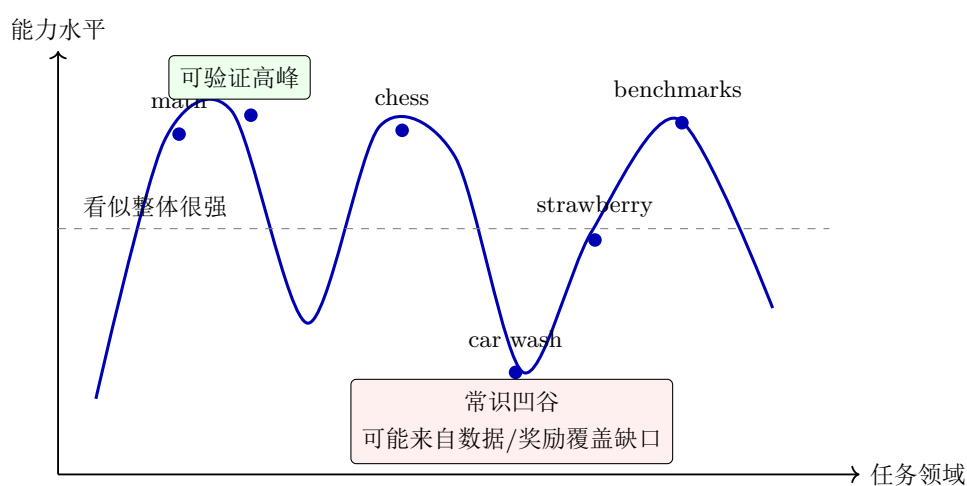


图 16: 锯齿能力分布示意图：能力曲线在可验证、训练覆盖充分的领域形成高峰，在分布外或奖励难构造的任务上出现凹谷。作者根据访谈内容绘制。

<sup>15</sup> 视频画面时间区间：00:10:34-00:10:45。





图 17: strawberry 字母计数与 50 米外洗车店路线选择等例子，用来说明模型能力分布并不平滑：相邻任务之间也可能落差很大。<sup>17</sup>

#### 4.4 数据分布与实验室偏好：为什么有些能力突然跃迁

锯齿智能还来自数据分布（data distribution）和实验室偏好。数据分布指训练数据覆盖的内容、风格、任务和频率组合。分布外（out-of-distribution / OOD）指当前任务落在训练数据、RL 环境或模型熟悉模式之外。一个任务是否分布外，并非看它对人类是否简单，而是看它是否被模型的训练经历充分覆盖。

Karpathy 提到 GPT-3.5 到 GPT-4 的国际象棋能力提升。很多人会把它理解成“模型整体推理能力自然变强”，但他强调另一个解释：大量 chess 数据进入预训练集后，模型在这一能力上形成了更明显的峰值。换句话说，某个能力跃迁有时来自训练混合物的调整。

##### 能力跃迁的三因素示意

某类任务上的能力提升，常可由三股力量共同解释：任务可验证、实验室愿意投入、训练数据覆盖充分。该关系是教学示意，不能当作论文定律。

直觉上，模型在某个任务上变强，需要能被反馈推动，也需要有人把任务放进训练系统。可写成下面的近似解释：

$$\Delta D_{\text{cap}}(\tau) \propto V(\tau) \cdot \lambda_{\text{lab}}(\tau) \cdot \text{Coverage}(\tau, D_{\text{train}})$$

- $\Delta D_{\text{cap}}(\tau)$ : 任务  $\tau$  上能力提升的幅度。
- $V(\tau)$ : 任务  $\tau$  的可验证性。
- $\lambda_{\text{lab}}(\tau)$ : 实验室对该任务投入数据、RL 环境和训练资源的程度。
- $\text{Coverage}(\tau, D_{\text{train}})$ : 训练数据分布对该任务的覆盖度。

<sup>17</sup>视频画面时间区间：00:11:45–00:11:58。

- $D_{\text{train}}$ ：训练数据分布。
- $\alpha$ ：表示“成比例的教学关系”，只用于解释方向，不表示精确物理定律。

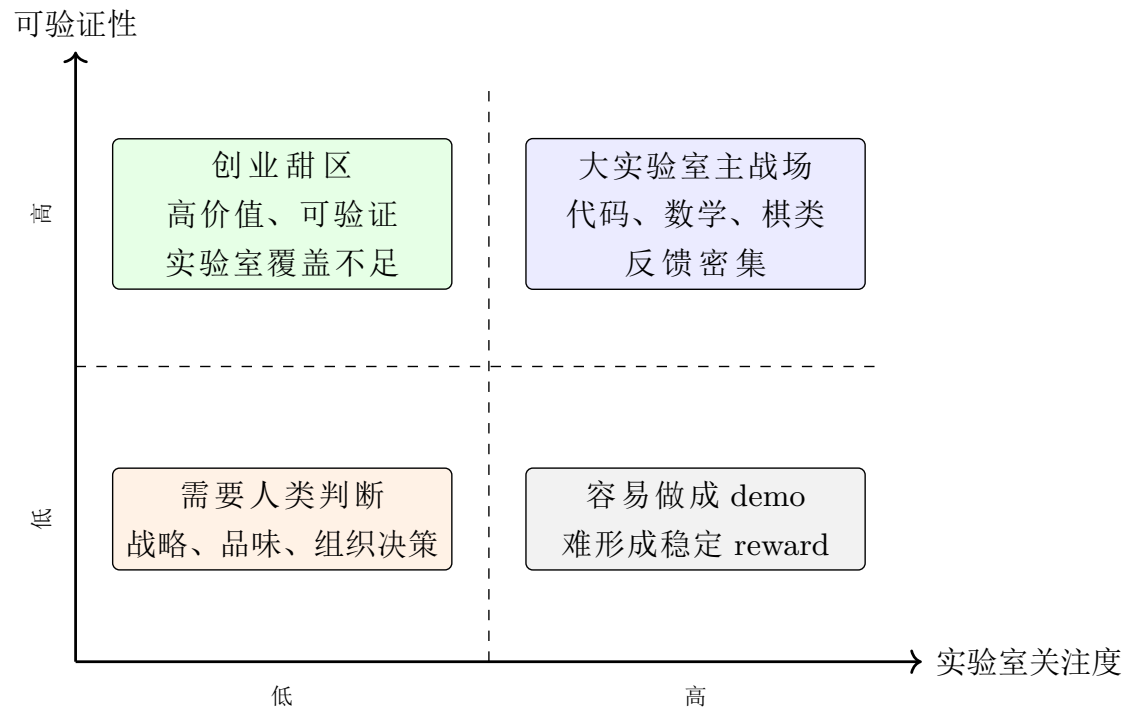


图 19: 棋类能力跃升的例子把本章推向数据分布与 RL 环境：实验室把哪些任务放进训练混合物，直接塑造模型的能力峰值。<sup>19</sup>

<sup>19</sup> 视频画面时间区间：00:13:38-00:13:39。

Karpathy 用 circuits 这个词描述模型内部被数据和 RL 强化出的可用能力路径。能力回路（circuits）在本讲义中作为比喻使用：如果应用落在被 RL 强化过、被数据覆盖过的回路里，模型表现会很顺；如果应用落在分布外回路，模型会挣扎。这里不需要把 circuits 过度神经科学化，它更像工程师说“这条路径已经被训练打通了”。

#### 应用开发者面对的是无说明书系统

前沿模型没有完整操作手册。团队必须通过实验确定自己的应用落在哪些能力回路里：在回路内可以借力飞行，在分布外需要自建数据、评估、微调或工作流约束。

## 4.5 本章小结

本章的因果链是：可验证性提供奖励，奖励让 RL 环境可扩展，可扩展环境塑造能力峰值，能力峰值与数据覆盖共同造成锯齿分布。传统自动化主要看任务能否写成代码，LLM 自动化更看输出能否被稳定验证。

strawberry、car wash、chess/RL data、circuits 四个例子共同说明同一件事：模型能力分布不平滑。它在数学、代码、漏洞挖掘等高可验证且高投入领域会快速抬升，在某些常识、意图理解或分布外任务上仍会失手。下一章的创业建议将从这里自然推出：高价值、可验证、实验室覆盖不足的任务环境，是小团队值得寻找的空间。

## 5 创始人该找什么：在“可验证但未被充分覆盖”的区域建环境

### 5.1 可验证性让问题变得 tractable

上一章已经把“可验证性”放在能力增长的中心：模型输出越容易被自动判好坏，越容易形成奖励，越容易被放进强化学习环境中反复改进。Karpathy 在给创始人的建议里，把这个技术规律翻译成了创业判断：创始人要找的领域，不能只停留在“AI 可以碰一碰”，还要能在当前范式下被持续推进。

这里需要补上一个创业语境中的关键术语：可处理性（tractability）。它指一个问题在当前技术路径下能否被有效推进，尤其取决于数据、验证、奖励和环境构造。一个问题商业价值很高，却缺少可判分反馈，团队就会陷入“做了很多 demo，却不知道有没有真正变好”的状态；一个问题只要能被稳定评分，就像游戏里终于出现了计分板，优化才有方向。

#### 关键结论

第 5 章的核心转换是：可验证性不只解释大模型为什么在某些能力上飞快进步，它也是创始人筛选机会的漏斗。能构造任务、能判分、能反复训练的垂直领域，才有机会从“用模型做功能”升级为“建立自己的能力优势”。

直觉上，可以把一个创业机会看成四层过滤。第一层是经济价值：客户是否真的愿意为结果付钱。第二层是可验证性：结果是否能被规则、测试、模拟器、模型评审或人工抽检稳定评价。第三层是环境可得性：团队是否能构造足够多样的任务和反馈。第四层是覆盖缺口：大实验室是否已经把这个领域当成重点训练对象。



图 20: Karpathy 在创业建议段落中回到 RL circuits 与 fine-tuning: 小团队的机会来自能构造任务、反馈和验证环境的垂直领域。<sup>21</sup>

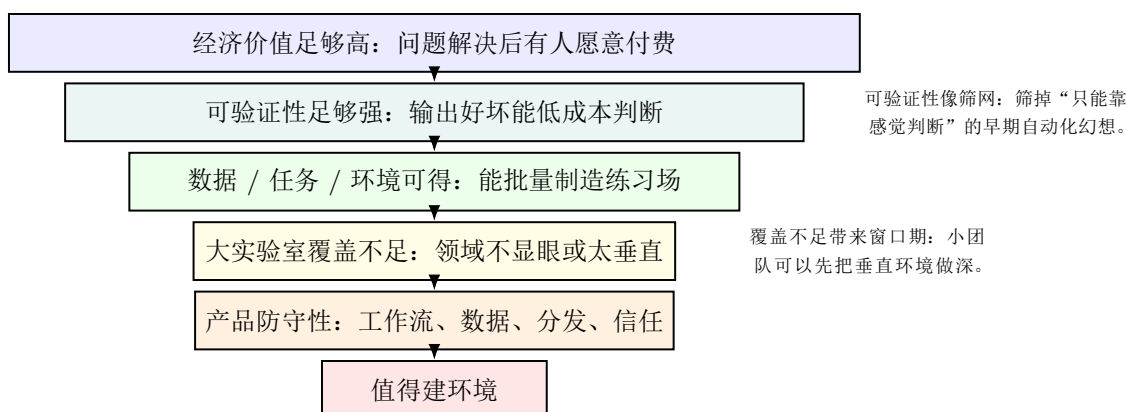


图 21: 创业机会筛选漏斗：从高价值问题开始，逐层检查验证、数据、环境和防守性，最后落到能被训练与评估的垂直环境。作者根据访谈内容绘制。

如果用一个教学公式压缩这条因果链，它表达的直觉是：一个领域是否值得创始人投入，不只看模型当前是否会做，还要看它能否被团队不断“喂反馈、造环境、拉开差距”。

以下符号只作本节启发式，不进入全篇符号表；它们服务于创业判断的直觉表达。

$$\text{Opportunity}(\tau) \uparrow \quad \text{when} \quad \text{Value}(\tau) \uparrow, V(\tau) \uparrow, \text{EnvBuild}(\tau) \uparrow, \lambda_{\text{lab}}(\tau) \downarrow$$

-  $\tau$ : 一个具体任务或垂直场景，例如合同审查、工业质检、医学编码、财务对账。-  $\text{Opportunity}(\tau)$ : 该任务对创业团队的机会强度。-  $\text{Value}(\tau)$ : 任务的经济价值，包含节省成本、增加收入、降低风险等收益。-  $V(\tau)$ : 任务的可验证性，沿用前文符号，表示输出能被稳定判好坏的程度。-  $\text{EnvBuild}(\tau)$ : 团队构造数据集、任务集、模拟器或 RL 环境的能力。-  $\lambda_{\text{lab}}(\tau)$ : 大实验室对该任务投入训练资源的程度，沿用前文“实验室关注权重”。

<sup>21</sup> 视频画面时间区间：00:14:08-00:14:15。



这个公式不能当成定量投资模型。它的用途是帮读者避免一个常见误判：看到基础模型在某个方向上很强，就以为创业机会已经消失。Karpathy 的意思更尖锐：只要该方向仍然可验证，且大实验室没有直接覆盖，创始人仍能通过环境和数据把“通用能力”压成“领域能力”。

5.2 创业机会：自建数据集、任务集与 RL 环境

Karpathy 的建议有一个很工程化的底色：如果创始人处在一个可验证场景里，能够创造 RL 环境或大量例子，那么团队可以拉动微调（fine-tuning）这个杠杆。微调指在基础模型上用领域数据或反馈继续训练，使模型更适合特定任务。它像给一个通才安排密集实战训练：基础能力来自预训练，领域熟练度来自后续的任务、答案和反馈。

这里的“自建”很关键。很多团队把 AI 产品理解成“调用最强模型 API，然后包一层 UI”。这种做法起步快，壁垒薄。更强的路径是把自己所在行业里的任务结构、评审标准、失败案例、边界条件沉淀下来，让它们变成模型可以反复练习的环境。数据集是静态教材，任务集是考试题库，RL 环境则像训练场：模型可以尝试、失败、拿到反馈、再尝试。

背景知识

数据集、任务集、RL 环境的差别可以用“书、考卷、训练场”类比。数据集告诉模型什么样的输入和输出经常一起出现；任务集让团队能比较不同模型和提示的表现；RL 环境进一步提供连续反馈，使模型或 agent 能在行动后观察结果并修正策略。

大实验室已经在数学、代码、竞赛题、网页任务等显眼场景里投入大量资源。创始人的空间往往藏在不那么“上镜”的垂直领域：验证逻辑清楚，但数据分散；任务价值很高，但外人看起来琐碎；错误成本明确，但需要行业流程才能构造判分器。比如保险理赔、发票异常检测、供应链合规检查、药物生产批记录审查、客服工单分派，这些领域的难点常常不在模型有没有语言能力，而在谁拥有真实任务分布和可靠验收标准。

这一点也解释了为什么“可验证但不显眼”比“听起来很宏大”更适合小团队。宏大任务容易变成愿景叙事，训练反馈稀薄；具体任务虽然窄，却能积累高密度反馈。对创业公司来说，窄并不丢人。窄意味着数据闭环更短、客户痛点更清楚、评估指标更硬。

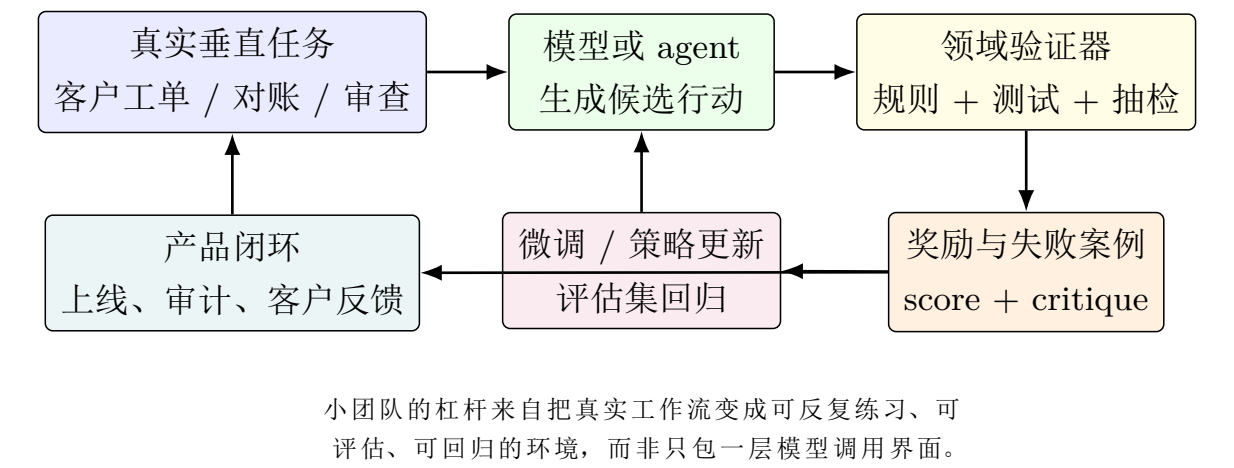


图 22: 垂直领域 RL 环境闭环：真实任务、模型输出、领域验证器、奖励、微调和产品反馈共同构成创业团队可积累的能力资产。作者根据访谈内容绘制。

可以把可验证创业机会写成一条因果链：



- 可验证性提供奖励信号。
- 奖励信号让任务可以被批量练习。
- 批量练习让团队构造 RL 环境或高质量微调集。
- 环境和微调集让模型在该垂直领域形成能力峰值。
- 能力峰值叠加产品流程、客户数据和分发渠道，才可能形成可防守业务。

### 注意

创始人需要警惕“验证器幻觉”。一个看似能打分的指标，如果和客户真正关心的结果偏离，就会把模型训练到错误方向。比如文档摘要的长度、格式完整度、关键词覆盖率容易被评分，但它们未必代表法律风险被正确识别。

## 5.3 自动化的远近：从确定性验证到 LLM judge

访谈中，主持人追问哪些事情仍然“远看才像能自动化”。Karpathy 的回答很直接：几乎所有东西最终都能在某种程度上变得可验证，只是难易程度不同。这个判断不应该被理解成“所有工作马上都会被完全自动化”。更准确的读法是：自动化边界（automation frontier）会沿着可验证性梯度移动，先吞掉容易判分的部分，再逐步逼近软任务。

自动化边界指当前技术能够可靠自动化与仍需人类深度参与之间的边界。它不像海岸线那样固定，更像潮水线：模型能力、工具环境、评估方法和人类监督流程都会改变它的位置。代码单测、数学判分、数据库一致性检查，离自动化中心很近；写作质量、战略判断、组织决策，距离更远，但也可以被拆出一些可验证子任务。

对于写作、评审、设计建议这类软任务，Karpathy 提到一种方法：模型评审（LLM judge）和多评审委员会（council of judges）。模型评审指用一个或多个模型对输出质量做评分、排序或审查。多评审委员会则把多个模型、多个 rubric 或多个视角组合起来，减少单一评审的偏差。

### 背景知识

LLM judge 可以看成“廉价但有偏的助教”。它能批量检查格式、覆盖点、明显逻辑漏洞、风格一致性，也能给出相对排序；它的问题在于偏好会漂移，容易被讨好，遇到高风险判断时还需要人类或确定性验证器兜底。

软任务的验证可以分层做。第一层是确定性约束，例如有没有引用未确认路径、有没有包含禁止占位符、有没有超出字数限制。第二层是结构性约束，例如论点是否覆盖 rubric，是否先动机再机制再例子。第三层是语义质量，例如是否准确、是否有洞见、是否适合目标读者。越往第三层，LLM judge 的价值越大，风险也越高。

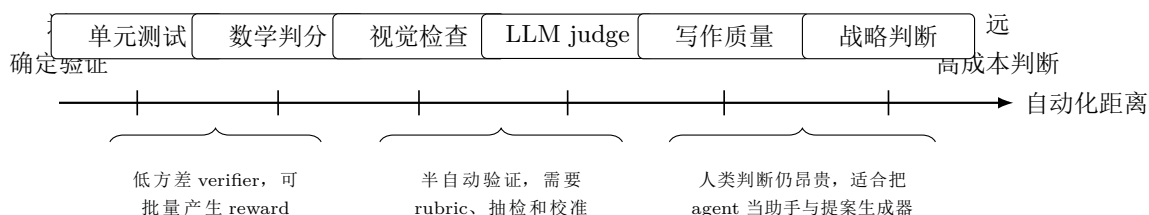


图 23: 自动化距离光谱：越靠左，reward 越容易构造，RL 环境越容易扩展；越靠右，验证成本、主观性和责任风险越高。作者根据访谈内容绘制。

因此，创始人需要把“自动化”拆成两个问题：第一，哪些局部判断已经足够可靠，可以自动放行；第二，哪些最终判断仍需要人类承担责任。前者决定效率，后者决定安全。把两者混在一起，会导致产品在 demo 中显得聪明，在生产中暴露不可控风险。

#### 关键结论

可验证性越高，自动化越靠近中心；可验证性越低，自动化越需要人类监督、抽检、申诉通道和责任归属。创业机会常常出现在中间地带：任务足够可验证，能显著自动化；任务又足够复杂，需要行业知识把验证器建对。

### 5.4 从环境到可靠交付

第 5 章还承担一个过渡任务：把“构造环境”连接到下一章的 agentic engineering。一个 agent 会做某件事，只说明模型能力已经出现；一个 agent 能稳定交付，才说明团队建立了工程闭环。环境和验证器提供的是第一层闭环：它们告诉模型输出是否接近目标。工程纪律提供第二层闭环：它检查安全、权限、回滚、可维护性和责任归属。

对创始人来说，这意味着产品路线不能停在“模型答得不错”。真正有价值的路线通常长这样：先找到可验证高价值任务；再把真实工作拆成可训练、可评估的任务单元；再积累失败案例和边界案例；再建立微调、评估、上线、回滚的循环。这样的公司卖的就超出了模型调用本身，核心资产会变成一个不断变强的任务环境。

#### 注意

“一切都可自动化”最危险的误读，是忽略自动化的可靠性等级。能生成一次正确答案、能在评测集上表现好、能在客户生产环境中持续稳定工作，是三个不同门槛。创业公司如果把第一层当成第三层，质量债会很快反噬。

### 5.5 本章小结

本章把上一章的可验证性理论转成创业判断。可处理性来自数据、验证、奖励和环境构造；微调只是杠杆，真正稀缺的是高质量任务分布和可靠反馈。创始人应寻找高价值、可验证、实验室覆盖不足的垂直场景，通过自建数据集、任务集和 RL 环境形成能力峰值。

同时，自动化边界会沿着可验证性梯度移动。确定性验证最容易自动化，写作和评审等软任务可以借助 LLM judge 或多评审委员会部分验证，但偏差、可操控性和责任归属必须被显式管理。下一章会进一步说明：当 agent 进入真实工程，核心问题从“能不能做”升级为“如何在速度提升时守住质量线”。

## 6 从 vibe coding 到 agentic engineering: 速度、质量与责任同时成立

### 6.1 vibe coding 提升地板

Karpathy 在这一段先给出一个清晰对照：vibe coding 提升了软件创作的地板。前文已经定义过 vibe coding，这里只回指它的行为模式：人用自然语言驱动软件创作，快速让模型生成、运行、修补，再继续迭代。它的意义在于让更多人跨过“从想法到可运行软件”的初始门槛。

“地板”这个词很重要。过去，一个非专业开发者可能连本地环境、依赖安装、数据库迁移、登录流程都跨不过去；现在，*agentic tools* 可以把这些细节托起来，让用户更早看到可交互的产品。对专业工程师来说，地板也在上升：样板代码、脚手架、测试补齐、API 查询、迁移脚本，越来越多低层劳动可以委派出去。

### 关键结论

vibe coding 的贡献是扩大“谁能做软件”和“多快能做出第一版”。它降低创作摩擦，提高探索速度，让软件从少数人的专业活动扩展为更多人的表达工具。

但地板上升不等于质量线自动上升。原型可以靠感觉推进，生产系统需要经受漏洞、安全、数据一致性、性能、审计和长期维护的压力。Karpathy 明确说，人仍然要对软件负责，不能因为用了 vibe coding 就引入漏洞。换句话说，模型让手变快了，责任没有消失。



图 24: Karpathy 解释 vibe coding 的“raising the floor”：更多人能把自然语言意图转成可运行软件原型。<sup>23</sup>

<sup>23</sup> 视频画面时间区间：00:16:01–00:16:08。

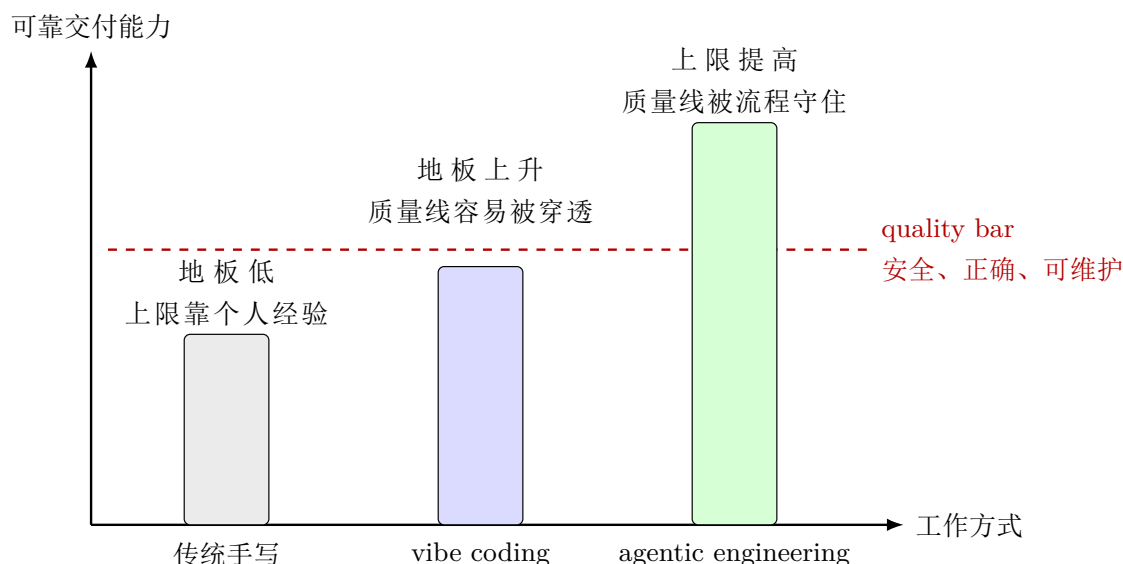


图 25: floor、ceiling 与 quality bar: vibe coding 扩大软件创作入口, agentic engineering 用 spec、测试、审查和权限边界把速度转成可靠交付。作者根据访谈内容绘制。

## 6.2 agentic engineering 维护质量线

本章首次完整定义 agentic engineering: agentic engineering (智能体工程) 是用工程纪律协调 agent, 在加速交付的同时维持安全、质量、可维护性和责任归属。它继承软件工程的严肃部分: 规格、测试、审查、部署、监控、回滚、权限、审计; 它新增的部分是如何管理一批强大、随机、尖峰化的 agent。

质量线 (quality bar) 指产品或代码进入生产环境前必须满足的安全、正确性、可维护性、性能和体验标准。它像桥梁工程里的载重线: 施工工具再先进, 桥仍然要承重。代码也是一样, 生成速度提高后, 质量线更需要显式化, 否则团队只会更快地产生更多不可靠产物。

这里可以用一个公式表达 agentic engineering 的基本约束。直觉是: 是否可以发布, 不能只看 agent 有没有完成任务, 还要看产物是否越过质量线、风险是否低于上限、人类判断是否通过。

$$\text{Ship}(y) = 1 \quad \text{only if} \quad Q(y) \geq q_{\min}, \rho(y, P, T) \leq \rho_{\max}, J_h(y, P) = \text{pass}$$

-  $y$ : agent 或团队产出的系统、代码、文档或部署结果。-  $Q(y)$ : 产物质量评分, 包含正确性、安全性、维护性和体验。-  $q_{\min}$ : 最低质量线。-  $\rho(y, P, T)$ : 在给定规格说明  $P$  和工具集合  $T$  下的风险。-  $\rho_{\max}$ : 可接受的最大风险。-  $J_h(y, P)$ : 人类基于理解、经验和设计品味做出的最终判断。-  $\text{Ship}(y)$ : 是否允许发布的决策, 1 表示允许发布。

### 注意

速度提升会放大团队的真实工程水平。成熟团队会把 agent 接入测试、审查和部署流程; 松散团队会把同样的速度变成漏洞、重复代码和隐性质量债。工具越强, 工程纪律越不能省。

Karpathy 还强调 agent 的两个性质: 随机性 (stochasticity) 和尖峰能力 (spiky capability)。随机性指模型输出具有概率性, 同样输入或相似输入可能产生不同结果。尖峰能力指 agent 在某些任务上表现极强, 在另一些邻近任务上突然失手。它是前文锯齿智能在工程侧的表现。

这两个性质共同改变了工程管理方式。传统程序的失败常常来自确定逻辑里的 bug; agent 的失败还可能来自采样、上下文缺口、错误工具调用、幻觉推断、评估盲区。工程师要做的工作, 就像给

一组高能但不稳定的执行体铺轨道：明确目标，限制权限，收集日志，用测试和审查约束输出，再把危险动作放进可回滚流程。

### 6.3 速度上限：从 10x 到更大的差距

Karpathy 提到，以前行业喜欢说“10x engineer”，但在 agentic engineering 里，优秀使用者和普通使用者之间的差距可能被进一步放大。这个说法不该被理解成固定倍数承诺。它表达的是一种结构性变化：当一个工程师能同时组织多个 agent、管理上下文、设计评估、快速合并结果，他的吞吐量不再只受个人打字速度和记忆能力限制。

10x 和 100x 的差别，关键不在单次 prompt 写得多漂亮，而在系统化委派。一个普通使用者可能把 agent 当成聊天框：问一个问题，拿一段代码，手动粘贴。一个强使用者会把任务拆成可并行子任务，给每个 agent 充分上下文，要求它们写测试、跑检查、互相审查，再由人统一合并。前者是“加速手指”，后者是“扩展工作台”。

#### 背景知识

“10x/100x”更适合作为能力差距的方向性描述，而非可审计的生产率指标。真正可观察的差距来自交付大项目时的端到端表现：任务分解是否清楚，质量线是否明确，agent 输出是否被验证，失败是否能回滚。

这种差距也会改变团队结构。过去，工程组织常按人力规模估算吞吐；未来，组织更需要看每个人能调度多少可靠 agent、能建立多少复用环境、能把多少流程变成可评估闭环。人类不再只是写代码的人，更像一套工作流的设计者和质量负责人。

### 6.4 AI-native 工程师的工作方式

AI 原生工程师（AI-native engineer）指把 agent、工具配置、上下文管理、测试与多任务委派融入日常工程方法的人。Karpathy 用编辑器类比这个变化：过去强工程师会深度配置 Vim、VS Code、shell、调试器；现在强工程师会深度配置 Claude Code、Codex、OpenCode、CI、测试脚手架、知识库和本地工具。

AI-native 的关键不只是“会 prompt”。它包含一组可训练的工程习惯：

- 把项目背景、架构约束、代码风格和测试命令写进可复用上下文。
- 把大任务拆成明确边界的子任务，减少 agent 互相踩文件和重复定义。
- 让 agent 先读代码和错误日志，再行动。
- 要求 agent 写或运行验证，避免只看生成文本。
- 用多 agent 做实现、审查、攻击和一致性检查。
- 把权限、密钥、部署动作放进受控流程，避免 agent 直接触碰高风险资源。



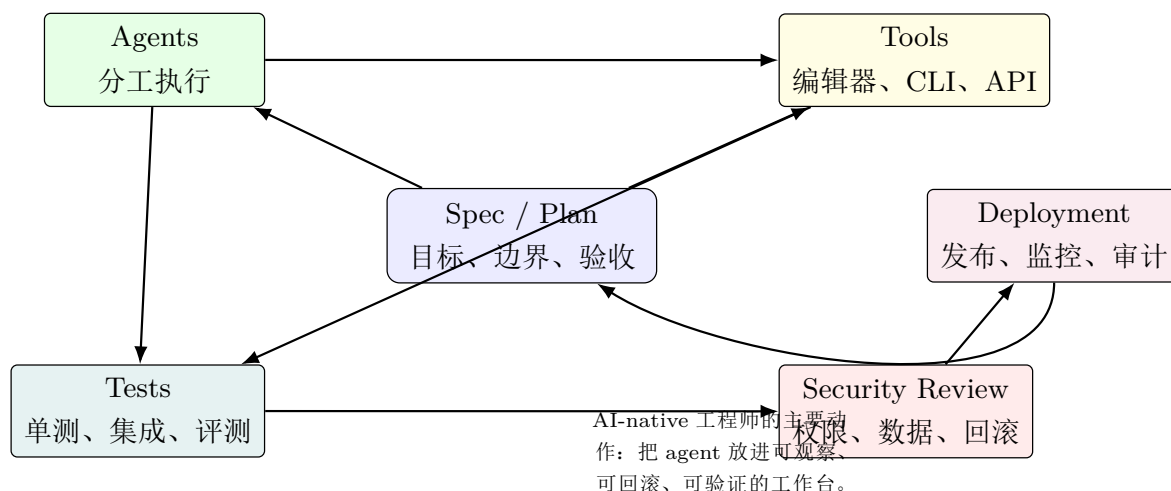


图 26: AI-native 工程工作台：agent 不单独决定生产质量，质量来自 spec、工具、测试、安全审查、部署反馈共同形成的约束系统。作者根据访谈内容绘制。

### 关键结论

AI-native engineer 的优势来自“环境配置 + 任务分解 + 评估闭环”。只会让模型生成代码的人，很快会被工具平均化；能把 agent 放进高质量工程系统的人，才会持续拉开差距。

这一点也解释了为什么 agentic engineering 会成为一门纪律。它有自己的失败模式：上下文污染、重复实现、误删文件、错误假设、测试覆盖假象、权限越界、评审疲劳。它也有自己的最佳实践：小步提交、独立工作区、明确边界、强制验证、对抗式审查、审计日志。传统软件工程没有过时，只是多了一个随机执行层。

## 6.5 招聘与评估：从谜题到大项目攻防

如果 agentic engineering 真的代表新的工程能力，招聘方式也必须跟着变。Karpathy 批评了仍然用小谜题、小算法题评估候选人的旧流程。原因很直接：这些题目测的是单点解题能力，无法观察候选人如何组织 agent、设定质量线、搭建测试、处理安全、交付完整产品。

他提出的招聘图景更接近真实项目：给候选人一个大项目，比如为 agent 构建一个 Twitter clone；要求它足够好、足够安全；再让一些 agent 模拟用户活动；最后用一批 attack agents 尝试破坏这个网站。候选人的表现不只体现在“能否写出功能”，还体现在能否经受攻击、能否修复、能否解释系统边界。

对抗式评估（adversarial evaluation）指用攻击者视角或专门 agent 尝试破坏系统，以检查安全和鲁棒性。它和传统安全红队、fuzzing、故障演练一脉相承，只是攻击者也可以被自动化放大。一个系统在友好 demo 中可用，不代表它在恶意输入、并发请求、权限绕过、数据污染和提示注入面前可靠。



图 27: 访谈进入 AI-native hiring: 评估重点从小算法题转向候选人如何借助 agent 交付大项目并承受攻防检验。<sup>25</sup>

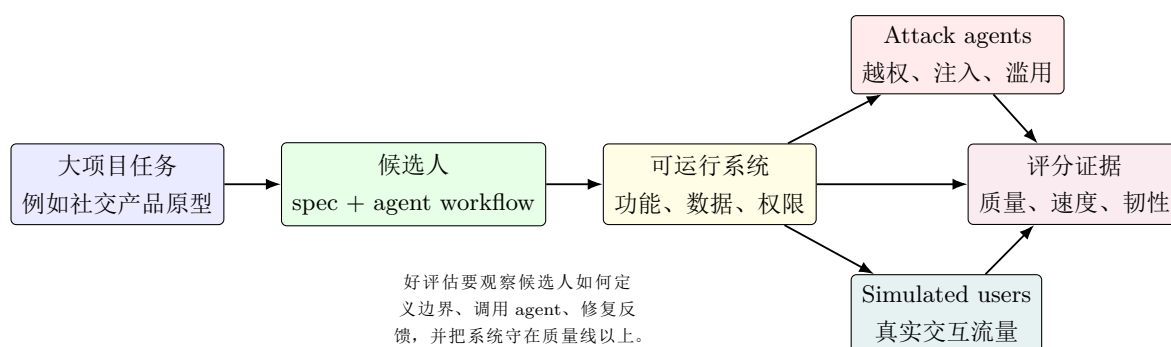


图 28: 招聘与攻防评估示意: AI-native 工程能力需要在大任务、模拟用户、攻击 agent 和质量门禁中观察。作者根据访谈内容绘制。

### 注意

小题面试会系统性低估 agentic engineering。它看不到候选人是否会写规格、约束权限、建立评估、处理部署、抵御攻击。它也会高估那些擅长一次性答题，却无法维护复杂系统的人。

招聘评估可以围绕四个问题展开。第一，候选人是否能把模糊目标转成可执行 spec。第二，候选人是否能主动调度 agent，并保持对输出方向的控制。第三，候选人是否能建立质量线，包括测试、安全审查和性能检查。第四，候选人是否能在 attack agents 暴露问题后，快速定位根因并做出可维护修复。

这类评估更贵、更慢，但更接近真实工作。直说就是：如果公司要招聘 agentic engineer，却仍然只考 45 分钟谜题，它测到的能力和岗位核心能力会严重错位。

<sup>25</sup> 视频画面时间区间：00:19:30-00:19:38。

## 6.6 本章小结

本章定义了标题中的关键升级：vibe coding 提升软件创作地板，让更多人更快做出可运行原型；agentic engineering 则用工程纪律协调 agent，在速度提升时守住质量线、安全性、可维护性和责任归属。agent 强大，但带随机性和尖峰能力，因此需要测试、审查、权限、日志、回滚和人类最终判断。

AI-native engineer 的核心能力超出提示词技巧，落在配置工具、组织上下文、拆分任务、并行委派、建立评估闭环上。招聘也应从小谜题转向大项目攻防：让候选人交付真实系统，再用模拟活动和 attack agents 检验鲁棒性。下一章会继续追问：当 agent 做得越来越多，人类到底还必须负责什么。

## 7 人类仍负责什么：spec、taste、fundamentals 与 agent-native 世界

沿用上一章对 vibe coding 与 agentic engineering 的定义，本章追问：当 agent 能写大量代码、查 API、部署服务、跑工具链时，人类还负责什么？

Karpathy 的回答很直接：人类仍负责目标、规格说明、设计品味、基本原理和监督。agent 很像一类记忆力极强、执行速度很快、偶尔会犯怪错误的实习生。它们能填很多空白，却需要人把边界画清楚，把身份模型、数据模型、权限模型和验收标准写成可执行的契约。

### 7.1 agent 像实习生：强记忆、弱判断、会犯奇怪错误

这段讨论的动机来自一个现实体验：agent 的能力分布有锯齿。它可能一分钟写完一批样板代码，也可能在用户身份绑定这种基础工程问题上犯错。这里的“实习生”类比主要强调三个工程事实。

第一，agent 的回忆能力很强。PyTorch、NumPy、Pandas 之间的 `keepdim`、`axis`、`dim`、`reshape`、`permute`、`transpose` 等 API 差异，已经很适合交给 agent 处理。第二，agent 的输出带随机性，同一类任务可能出现质量波动。第三，agent 缺少稳定的产品判断和系统边界意识，尤其容易在“看起来能跑”的实现里埋下身份、权限、隐私、账务和维护性问题。

#### 关键结论

本章采用的工作定义：规格说明（spec）是对目标、约束、边界、数据模型、验收标准和非目标的明确描述。它比 prompt 更硬，也比临时计划更完整。prompt 像一句任务委托，spec 像一份行动契约。

用符号看，spec 可以写成一个由目标、边界、验收标准和非目标组成的对象。直觉上，它回答四个问题：要什么、不能碰什么、怎样算完成、哪些诱人的方向先排除。

$$P = (G, B, A_{\text{accept}}, N)$$

-  $P$ ：规格说明（spec），给 agent、工程师和评审者共同使用的行动契约。-  $G$ ：目标（goals），说明系统要实现的用户价值或业务结果。-  $B$ ：边界（boundaries），说明数据、权限、架构、时间和风险限制。-  $A_{\text{accept}}$ ：验收标准（acceptance criteria），说明怎样检查产物已经达标。-  $N$ ：非目标（non-goals），说明本轮明确不解决的事项，防止范围蔓延。

当 agent 只拿到一句“实现支付积分”时，它会从局部线索里猜系统模型。当它拿到完整 spec 时，它才有机会把账务、身份、权限和异常路径纳入同一个设计空间。

## 7.2 spec 与 design taste: 从用户 ID 错误到系统边界

Karpathy 提到 MenuGen 中的一个 bug: 用户用 Google 账号登录, 同时通过 Stripe 购买 credits。Google 账号和 Stripe 账号都有 email。agent 把 Stripe email 和 Google email 当成同一用户的关联依据, 结果用户只要在两个系统里使用不同 email, 购买的 credits 就无法正确归到登录用户身上。

这类故障不能缩小成“email 字符串比对写错了”的小问题。它暴露的是身份模型缺失。email 是可变的、可重复出现于不同系统语境的联系信息; 持久用户 ID (persistent user ID) 才应该是跨登录、支付、权限、账务等系统稳定识别同一用户的主键。Google email 和 Stripe email 都只能作为外部属性, 不能替代系统内部的用户主键。

### 注意

Stripe email / Google email 错绑的根因是 spec 没有显式规定身份模型。正确 spec 至少应写清: 系统内部以 `user_id` 为主键; Stripe customer、payment intent、credit ledger 通过 `user_id` 关联; email 只用于通知、展示或辅助查找; 任何资金归属都不得依赖跨服务 email 相等。



图 29: Karpathy 用 email 绑定错误说明 agent 的“怪异”失败模式: 看似合理的捷径会破坏身份模型与资金归属。<sup>27</sup>

<sup>27</sup> 视频画面时间区间: 00:20:05–00:20:14。

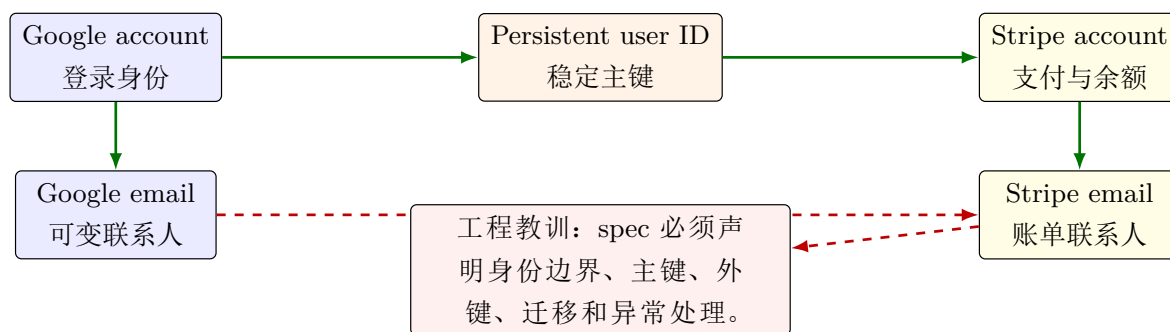


图 30: 身份绑定错误图: email 是可变属性, persistent user ID 才能承担跨登录、支付、权限和资金归属的稳定身份职责。作者根据访谈内容绘制。

设计品味 (design taste) 在这里也登场。它覆盖简洁性、抽象边界、用户体验、架构协调和长期维护成本, 范围远大于 UI 审美。好的品味会问: 这个主键能否跨系统稳定存在? 这个抽象有没有把业务事实藏起来? 这个实现通过了 happy path 后, 是否会在账务、权限、并发或回滚上出事?

人类判断可以被看成一道最终闸门。直觉上, 测试能验证已写明的条件, 设计品味要补上那些还没有被机器稳定奖励的质量维度。

$$\text{Ship}(y) = 1 \quad \text{only if} \quad Q(y) \geq q_{\min}, \rho(y, P, T) \leq \rho_{\max}, J_h(y, P) = \text{pass}$$

-  $y$ : agent 或团队产出的代码、系统、文档或部署结果。-  $Q(y)$ : 产物质量评分, 覆盖正确性、安全性、维护性、性能和体验。-  $q_{\min}$ : 最低质量线, 低于它就不应进入生产环境。-  $\rho(y, P, T)$ : 在 spec  $P$  和工具集合  $T$  下的工程风险。-  $\rho_{\max}$ : 团队可接受的最大风险。-  $J_h(y, P)$ : 人类基于理解、经验和设计品味做出的最终判断。

### 7.3 API 细节可以外包, 底层模型不能外包

Karpathy 的张量例子非常关键。他说自己已经不想记住很多 API 细节: 到底是 `keepdim` 还是 `keepdims`, 某个库叫 `dim` 还是 `axis`, 该用 `reshape`、`permute` 还是 `transpose`。这些细节像城市里的路名, agent 查得快、记得准、试得勤。

但人不能放掉底层模型。张量视图 (tensor view) 是与底层 storage 共享数据、只用不同形状、步幅或维度解释同一块数据的张量表示。storage 是实际存放数据的内存区域。view 像同一本书的不同目录视角, storage 像书页本身。改变目录视角通常便宜, 复制整本书就昂贵。

#### 背景知识

张量里的 `reshape`、`transpose`、`permute` 往往涉及“视图是否仍能共享 storage”这个底层问题。若一次操作迫使内存复制, 模型训练或推理可能在不显眼的地方变慢、变贵、变得更难调试。agent 可以查 API, 人类仍要知道它在移动指针、改变 stride, 还是复制内存。

直觉上, 底层理解的作用是让人能区分“语法能跑”和“机制合理”。可以用一个简化关系表示 view 与 storage。

$$\text{view} = \text{Interpret}(\text{storage}, \text{shape}, \text{stride}, \text{offset})$$

- view: 张量视图, 以某种形状和步幅解释数据。- Interpret: 解释操作, 表示同一块底层数据可以被不同方式读取。- storage: 底层存储, 实际保存数值的内存区域。- shape: 张量形状, 例如行数、



列数和高维尺寸。- stride: 步幅, 说明沿每个维度移动一步时底层存储索引如何变化。- offset: 起始偏移, 说明视图从 storage 的哪个位置开始读取。

这个例子延展到所有 agentic engineering: 人可以让 agent 处理语法、样板、迁移和查询; 人仍要掌握数据结构、系统边界、性能模型、安全模型和失败模式。没有这些 fundamentals, 人会失去验收能力, 只能被 agent 的流畅输出牵着走。

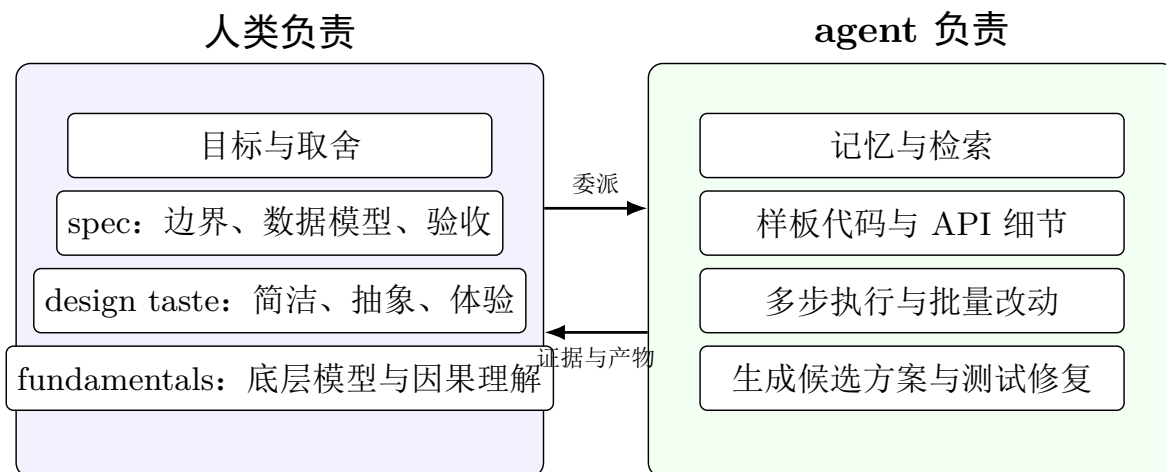


图 31: 人类 / agent 分工: API 细节和样板执行可以交给 agent, 目标、spec、设计品味和底层理解仍由人类承担。作者根据访谈内容绘制。

## 7.4 为什么美学和简洁仍在 RL 回路之外

Karpathy 对 agent 代码的担忧也很具体: 代码可能臃肿, 复制粘贴很多, 抽象别扭且脆弱。它能够通过测试, 却让人读起来心里发紧。这个现象可以用前文的可验证性解释: 很多训练和评估环境会奖励“能跑”“能通过测试”“能修复报错”, 但简洁、优雅、少冗余、好抽象这些质量很难被稳定打分。

美学奖励 (aesthetic reward) 就是对简洁、优雅、少冗余、好抽象等质量的奖励信号。它当前仍不充分, 因为这些维度往往依赖长期维护视角、项目语境、团队风格和未来变化。一个函数今天通过测试, 半年后是否还能自然扩展, 这类问题很难在短回合 RL 环境里自动判定。

### 关键结论

设计品味暂时仍是人类高价值能力, 因为它站在更长时间尺度上判断系统。测试关注眼前行为是否正确, taste 关注结构是否经得起修改、协作、迁移和规模扩大。

microGPT 简化案例说明了这个缺口。Karpathy 希望把 LLM 训练代码压到尽可能简单, 反复要求模型继续简化, 却发现模型很难进入那种“越少越清楚”的目标。用前文术语说, 这像是落在当前能力回路之外: 模型有大量会写代码的模式, 却缺少足够强的“极简教学实现”奖励。

直觉上, 美学奖励可以作为质量函数的一部分, 但它难以被完全自动化。它更像人的判断与工具评估的混合项。

$$Q(y) = \alpha R_{\text{test}}(y, A_{\text{accept}}) + \beta A_{\text{taste}}(y) + \gamma R_{\text{maintain}}(y)$$

-  $Q(y)$ : 产物总体质量。-  $y$ : 被评估的代码或系统。-  $\alpha, \beta, \gamma$ : 不同质量维度的权重, 取决于团队目标和风险环境。-  $R_{\text{test}}(y, A_{\text{accept}})$ : 产物相对验收标准的测试或检查结果。-  $A_{\text{taste}}(y)$ : 设计品味评

价，关注简洁、抽象边界、体验和架构协调。-  $R_{\text{maintain}}(y)$ ：维护性评价，关注可读性、可扩展性、局部性和长期修改成本。

这个公式只是教学示意，不能当成可直接部署的评分器。它的价值在于提醒读者：通过测试只是质量的一部分。

## 7.5 agent-native 基础设施：面向 agent 的文档、权限、传感器与执行器

访谈后半段从个人编码推进到基础设施。Karpathy 说，现在大多数框架、服务和部署平台仍主要写给人类使用。文档告诉人去哪个页面、点哪个菜单、复制哪个值、配置哪个 DNS。可是在 agentic workflow 中，人真正想要的是一段能交给 agent 执行、可检查、可回滚的操作契约。

agent-native 基础设施（agent-native infrastructure）指面向 agent 行动设计的文档、API、权限、审计、数据结构和部署流程。它要求服务把“给人看的说明”升级为“给 agent 的行动接口”：步骤清晰、状态可读、权限明确、错误可恢复、日志可审计。

这里可以用传感器（sensors）和执行器（actuators）来理解。传感器是 agent 读取世界状态的数据入口，例如文件、网页、日志、数据库、日历、构建结果、支付状态。执行器是 agent 改变世界状态的行动出口，例如写文件、发 API 请求、部署、发邮件、开权限、改 DNS。中间必须有权限边界和审计日志，否则 agent 的一次错误行动可能直接影响用户资金、生产服务或隐私数据。

### 注意

agent-native 不等于“给 agent 所有权限”。更可靠的形态是：传感器尽量丰富，执行器分级授权，高风险动作需要确认或沙箱演练，所有副作用写入审计日志，并且有回滚路径。



图 32: Karpathy 讨论 agent-native infrastructure 时提出 sensors 与 actuators 类比：系统需要把可读取状态与可执行动作清楚拆开。<sup>29</sup>

<sup>29</sup> 视频画面时间区间：00:26:14–00:26:22。

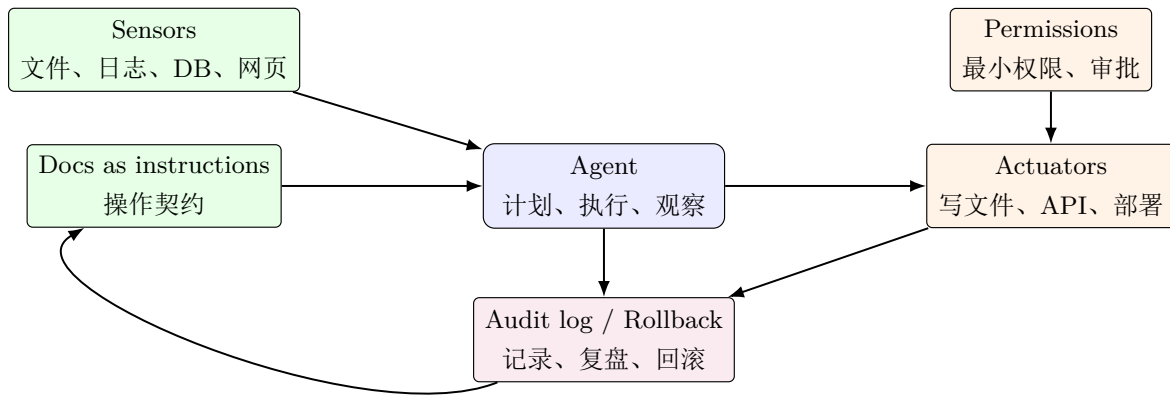


图 33: agent-native 基础设施：面向 agent 的文档、传感器、执行器、权限和审计共同限定“能看到什么、能做什么、做错如何回滚”。作者根据访谈内容绘制。

直觉上，agent 每一步行动都应被权限边界约束，并被审计日志记录。

$$a_t \in \Pi(T), \quad L_{\text{audit}} = \{(t, a_t, \text{obs}_t)\}_{t=1}^k$$

-  $a_t$ : agent 在第  $t$  步采取的行动。-  $T$ : agent 可用的工具集合。-  $\Pi$ : 权限边界，规定哪些工具、资源和副作用在当前任务中被允许。-  $\Pi(T)$ : 在权限边界内可调用的工具子集或行动集合。-  $L_{\text{audit}}$ : 审计日志。-  $\text{obs}_t$ : 第  $t$  步行动后的观察结果，例如命令输出、API 响应、部署状态或错误信息。-  $k$ : 本轮任务的行动步数。

MenuGen 的部署困扰就是一个好测试：如果“构建 MenuGen 并部署到线上”仍要求人反复进入 Vercel、DNS、支付平台和环境变量页面点击配置，那么基础设施仍偏 human-facing。若 agent 能读取标准化文档、申请最小权限、完成部署、展示 diff、生成审计记录，并在失败时回滚，基础设施才接近 agent-native。

## 7.6 本章小结

本章把“agent 越来越能干”翻译成一个更硬的工程判断：agent 能填空，人类要定义空格的边界。

第一，spec 是 agentic engineering 的行动契约。Stripe email / Google email 错绑说明，身份模型若没有写入 spec，agent 会用局部字符串相等替代系统主键，最终造成资金和用户关系错配。

第二，design taste 是当前仍难自动化的质量能力。模型擅长让代码通过测试，但简洁、抽象边界、长期维护和产品体验仍需要人类判断。

第三，API 细节可以交给 agent，fundamentals 不能丢。张量 view/storage 的例子说明，人可以忘记库函数细节，但要理解内存、数据结构、性能和语义。

第四，agent-native infrastructure 会把文档、部署、权限和审计改写成面向 agent 的行动接口。传感器让 agent 看见世界，执行器让 agent 改变世界，权限和审计决定这个世界能否承受自动化。

## 8 综合章：廉价智能时代，不能外包的是理解

最后一段讨论回到教育。Karpathy 引用了一句让他反复思考的话：人可以外包思考过程，但不能外包理解。这里的“思考过程”指搜索、推导、草拟、生成、对比、试错等可被模型大量加速的过程性劳动；“理解”指人对目标、结构、因果关系、取舍和边界形成的内在模型。

这也是全片的落点。模型能力增长得很快，智能变便宜，执行变廉价，但系统里仍需要一个能决定方向、提出问题、判断结果、修正路线的人。这个人类角色就是指挥者（director）。

## 8.1 你可以外包思考过程，但不能外包理解

Karpathy 说自己仍然是系统的一部分，而且越来越像瓶颈：信息必须进入他的脑子，他需要知道要构建什么、为什么值得构建、怎样指挥自己的 agents。廉价智能让过程膨胀得很快，人的理解若跟不上，委派会变成失控的产出堆积。

### 关键结论

理解（understanding）是人对目标、结构、因果关系、取舍和边界的内在模型。它决定人能否写出好 spec、识别风险、判断设计品味，并把 agent 的大量输出转化成可靠行动。

外包思考过程（outsource thinking）是把推导、搜索、草拟、生成、试错等过程性交给模型完成。它非常有价值，但它产生的是候选答案、路径和材料。理解需要人把这些材料吸收进自己的结构模型里。否则，人会拥有更多文本、更快代码和更多建议，却没有更清楚的判断。

直觉上，agentic workflow 中最重要的闭环要落到“人类理解更新”，不能停在“模型输出结束”。模型产出应回流到人脑中的结构模型，下一轮 spec 才会更好。

$$U_h \rightarrow P \rightarrow C \rightarrow M_W \rightarrow y \rightarrow J_h \rightarrow U'_h$$

-  $U_h$ ：人类当前理解，包含目标、因果、边界和取舍。-  $P$ ：由理解生成的规格说明。-  $C$ ：交给模型的上下文，包括任务、资料、工具结果和约束。-  $M_W$ ：由权重  $W$  参数化的模型。-  $y$ ：模型产出，例如代码、解释、计划、设计或部署结果。-  $J_h$ ：人类判断和反馈。-  $U'_h$ ：经过产出、反馈和反思后更新的人类理解。

这条链解释了为什么教育仍重要。教育的目标不只是让人记住更多步骤，也不是让人和模型比赛 API 细节。它要训练人形成可迁移的内在模型：看到一个输出，知道它哪里可靠；看到一个系统，知道它缺哪个边界；看到一个机会，知道它靠什么机制成立。

## 8.2 知识库与投影：用 LLM 增强理解

Karpathy 对 LLM knowledge bases 很感兴趣，因为知识库可以帮助他处理信息。这里的知识库（knowledge base）是对资料进行组织、索引、重排和问答的外部记忆系统。它可以是 wiki、RAG 系统、个人笔记库，也可以是围绕一批文章、代码、访谈和实验日志建立的交互式资料层。

信息投影（projection）是把同一份信息从不同角度重新呈现，例如时间线、概念图、问答、对照表、反例清单、依赖图、公式化摘要。它和前文的信息重编译互相呼应：信息重编译强调把材料转成新形态，投影强调从一个固定资料集合里生成多种视角。

### 背景知识

同一份资料像一个三维物体。单篇文章是正面视图，时间线是侧面视图，概念图是俯视图，问答是局部放大镜。LLM 的价值在于快速生成这些投影，让人从多个角度校正自己的内在模型。

Karpathy 把这类操作称作在固定数据上做 synthetic data generation。它的重点是在同一批资料上生成新的阅读路径，前提是不能凭空编造事实。读一篇文章后，让 wiki 生成问题、对照表或概念图，人的理解可能因此突然抓住结构。

直觉上，知识库增强理解的过程可以写成“固定资料，多重投影，人类模型更新”。

$$\{z_i\}_{i=1}^n = \text{Project}(D_{\text{fixed}}, \{v_i\}_{i=1}^n), \quad U'_h = \text{Update}(U_h, \{z_i\}_{i=1}^n)$$

-  $D_{\text{fixed}}$ : 固定资料集合，例如文章、字幕、代码、论文或项目文档。  
 -  $v_i$ : 第  $i$  个视角，例如时间、概念、因果、对比、问题或行动清单。  
 - Project: 信息投影操作，把同一资料按指定视角重新呈现。  
 -  $z_i$ : 第  $i$  个投影结果。  
 -  $n$ : 投影视角数量。  
 -  $U_h$ : 投影前的人类理解。  
 -  $U'_h$ : 吸收投影后更新的人类理解。  
 - Update: 人类通过阅读、提问、纠错和连接已有经验来更新内在模型。



图 34: 教育段落的关键 takeaway: 可以把搜索、草拟、试错等思考过程交给模型，但理解必须被人吸收并内化。<sup>31</sup>

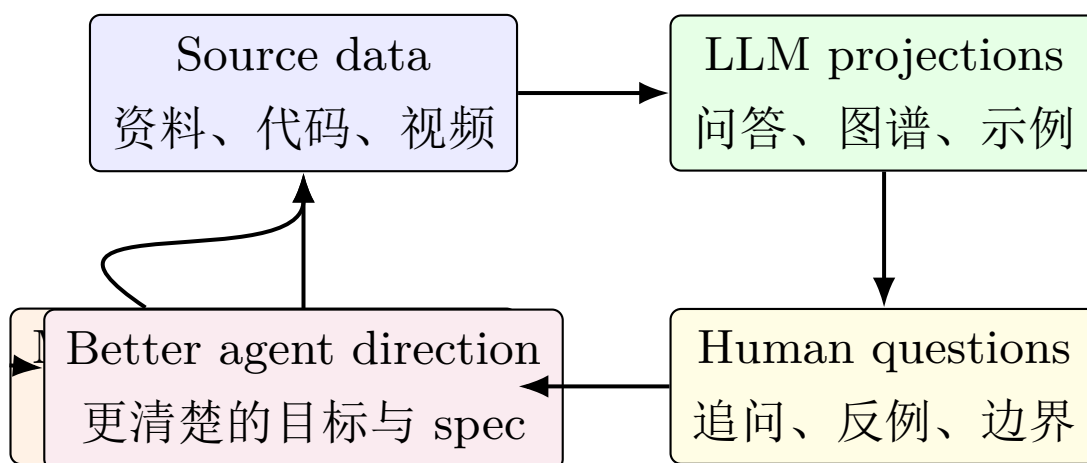


图 35: 理解增强闭环: LLM 可以把固定资料投影成多种视图，人类通过提问、反例和取舍更新心智模型，再用更好的 spec 指挥下一轮 agent 工作。作者根据访谈内容绘制。

<sup>31</sup> 视频画面时间区间: 00:28:14-00:28:23。



### 8.3 两条主线的汇合：verifier 让模型变强，spec 让团队变强

全片有两条主线。第一条是模型能力如何增长：软件 3.0 把上下文窗口变成新的程序接口；多模态模型让非结构化信息也能被重编译；可验证性提供奖励；奖励让 RL 环境可扩展；可扩展环境塑造能力峰值；数据覆盖和实验室关注共同造成锯齿智能。

第二条是人类如何把这种能力变成可靠生产力：vibe coding 让更多人能做软件；agentic engineering 要守住质量、安全和维护性；守住质量靠 spec、tests、review、taste 和 fundamentals；agent-native 基础设施把工具、权限、文档、部署和审计改造成可委派的行动环境；教育最终服务于理解，因为理解决定人能否成为好 director。

#### 关键结论

全片的压缩句：可验证性决定模型在哪些地方飞起来；spec 和 design taste 决定人类能否把飞起来的能力变成可靠产品。

这两条线在“验证”这个概念上相遇。对模型训练来说，验证器把输出质量变成奖励，推动能力上升。对工程团队来说，spec 把目标和边界变成可检查的验收标准，推动协作稳定。一个强化模型，一个强化团队。

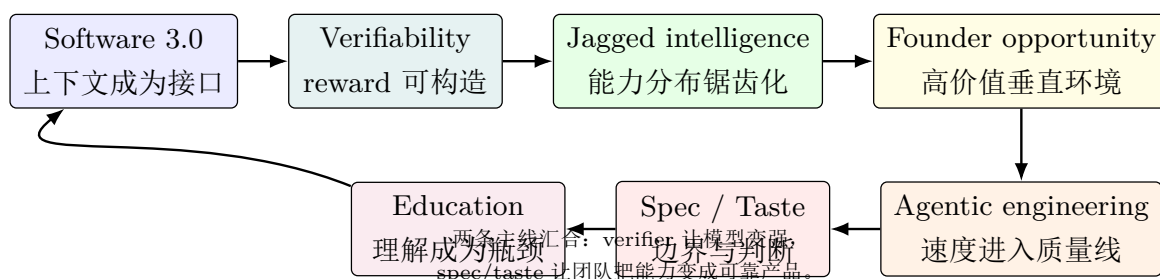


图 36: 全篇总图：从 Software 3.0 到理解瓶颈，整场访谈把模型能力增长、创业机会、工程纪律和教育目标串成一条因果链。作者根据访谈内容绘制。

用一个教学示意公式表达两条线的并行关系：模型侧靠验证器产生奖励，团队侧靠 spec 产生可验收的行动。

$$\text{Model improvement} \Leftarrow R(\tau, y), \quad \text{Team reliability} \Leftarrow R_{\text{test}}(y, A_{\text{accept}}) \text{ under } P$$

- Model improvement: 模型能力提升，尤其发生在可验证、可扩展训练的任务上。-  $R(\tau, y)$ : 验证器对任务  $\tau$  和输出  $y$  给出的奖励或评分。-  $\tau$ : 任务。-  $y$ : 模型或团队产出的结果。- Team reliability: 团队交付的可靠性。-  $R_{\text{test}}(y, A_{\text{accept}})$ : 根据验收标准对产物进行测试或检查。-  $A_{\text{accept}}$ : spec 中的验收标准。-  $P$ : 规格说明，规定目标、边界、验收和非目标。

这里还有一个容易忽略的反向概念：没有验证器，模型训练会缺少清晰反馈；没有 spec，团队协作会缺少共同标准。廉价智能放大了执行，模糊目标也会被放大。

### 8.4 speaker closing discussion: 教育、瓶颈与未来图景

访谈的实质性收尾集中在教育问题上。主持人问：当智能变便宜，下一代 AI 时代还有什么值得深学？Karpathy 的回答可以拆成三层。

第一，思考过程可以被大量委派。搜索资料、草拟代码、生成解释、提出候选方案、重排文章，都会越来越便宜。第二，理解仍必须进入人的脑子，因为人还要决定建什么、为什么建、如何指挥 agents。

第三，LLM knowledge bases 是一种增强理解的工具，因为它们能围绕固定资料生成多种投影，让人更快看到结构。

他也提到一个开放未来：也许几年后，系统会进一步自动化，人类是否还会被自动化出理解环节仍是问题。但在当前阶段，LLM 并不擅长替人承担最终理解。人仍独特地负责吸收、判断、指挥和承担后果。

### 注意

“智能变便宜”容易诱发一个误判：把产出数量当成理解深度。真正的 director 不以生成了多少计划衡量自己，而以能否提出更清楚的问题、写出更稳的 spec、识别更隐蔽的风险来衡量自己。

routine sign-off 可以删去，因为它只是在访谈礼貌性结束。值得保留的是教育判断：工具越强，人越需要理解目标、边界和取舍。

## 8.5 面向后续实践的清单

下面把全片收束成一份可执行清单。它把模型能力主线和工程实践主线压成可以立刻使用的工作方法，避免停留在口号层面。

### 一、判断任务是否适合交给模型加速。

- 问清任务的可验证性：输出能否被测试、规则、模拟器、模型评审或人工流程稳定检查。- 找出训练覆盖和实验室关注：它更像 math/code 这类高覆盖高奖励领域，还是垂直、冷门、分布外任务。- 预估锯齿风险：把“模型在相邻任务上很强”与“本任务一定可靠”分开看。

### 二、把委派写成 spec。

- 写目标：用户价值、业务目标或学习目标是什么。- 写边界：数据、权限、性能、安全、隐私、时间和成本限制是什么。- 写验收：哪些测试、样例、评审或观测能证明完成。- 写非目标：本轮明确不做什么，防止 agent 自动扩张范围。- 写身份和权限模型：所有账户、账务、用户、组织、资源都要有稳定主键和最小权限原则。

### 三、用 agentic engineering 守住质量线。

- 把 agent 当成高吞吐执行体，给它上下文、工具和检查点。- 对高风险行动设置确认、沙箱、回滚和审计日志。- 让测试覆盖已知要求，让人类 review 覆盖 taste、架构、安全和长期维护。- 对关键系统安排对抗式评估，让另一个 agent 或人工角色尝试破坏它。

### 四、保留 fundamentals。

- API 细节交给 agent 查询，底层模型留在人脑中。- 对张量、数据库、网络、并发、权限、身份、部署、缓存等基础概念保持机制级理解。- 每次被 agent 快速带过的地方，都问一句：这里实际改变了什么状态，复制了什么数据，暴露了什么权限，承担了什么风险。

### 五、训练 design taste。

- 不只问代码能否通过测试，也问它是否简洁、局部、可读、可扩展。- 对复制粘贴、脆弱抽象、过度泛化、隐藏耦合保持敏感。- 把优秀实现、失败案例和重构前后对比放进团队知识库，让 taste 有材料可训练。

### 六、用知识库增强理解。

- 把文章、会议、代码、实验、设计文档沉淀成可检索资料。- 对同一批资料生成多种投影：时间线、概念图、问答、反例、行动清单、依赖关系。- 用投影来更新自己的  $U_h$ ，再把更新后的理解转成更好的 spec。

### 七、以 director 的标准要求自己。

- 决定做什么和为什么做。- 分解任务并设置检查点。- 判断 agent 产出是否可信。- 在速度、风险、质量和学习之间做取舍。- 对最终结果负责。

## 8.6 本章小结

综合章把全片压回一个核心判断：廉价智能会降低执行成本，却会抬高理解的重要性。人可以把搜索、草拟、推导、试错交给模型，但不能把目标、因果、边界、取舍和责任一起交出去。

第一条主线解释模型：可验证性带来奖励，奖励放大 RL 环境，环境塑造能力峰值，数据覆盖和实验室投入造成锯齿智能。

第二条主线解释工程：vibe coding 抬高地板，agentic engineering 守住质量线，spec、taste、fundamentals 和 agent-native 基础设施把随机而尖峰的 agent 能力变成可交付系统。

最后的教育答案是：深学仍值得，因为理解是 director 的操作系统。没有理解，人只能消费模型输出；有了理解，人才能提出正确问题、搭建正确环境、委派正确任务，并判断什么值得进入现实世界。